

ESTRUTURAS DE DADOS

com Python e MySQL

Guia de estudo e consulta para os exemplos e atividades do pacote

Eixo	Como o material trabalha
Memória	tuple, list, dict, set e estruturas clássicas
Persistência	MySQL como fonte de dados; alterações só persistem com SQL explícito
Domínio	rede varejista didática com lojas/CDs, estoque, pedidos, reposição e rotas
Prática	167 exemplos completos + 288 atividades atômicas

REGRA CENTRAL — Um SELECT materializa valores na memória. Alterar uma estrutura Python não altera automaticamente o MySQL. Persistência exige SQL apropriado e, quando aplicável, commit().

Este material foi escrito para permanecer aberto durante o estudo dos arquivos em exemplos/ e a resolução dos arquivos em exercícios/. Ele não fornece soluções das atividades; fornece conceitos, critérios de escolha, explicações dos recursos e estratégias de raciocínio.

Como usar este material

A sequência acompanha exatamente as 24 pastas do pacote. Em cada capítulo, o recurso aparece primeiro como conceito de programação e depois é relacionado ao banco lojas_rede quando isso acrescenta significado. Os exemplos completos permanecem nos arquivos .py; aqui são reproduzidos apenas trechos selecionados para explicar o que observar.

Quando você...	Use o material assim
não entendeu um exemplo	leia a explicação do recurso, abra os exemplos citados e tente prever a saída antes de executar
travou em uma atividade	volte à seção “Como pensar nas atividades”; ela oferece perguntas e critérios sem entregar a solução
esqueceu a sintaxe	use a tabela “Consulta rápida” do capítulo
não sabe qual estrutura escolher	compare as seções “Quando usar” e “Evite quando”
não sabe se o banco mudou	procure a caixa “Memória x persistência” e confirme com um novo SELECT

Como estudar um exemplo

- Leia o nome do arquivo e tente antecipar qual comportamento será demonstrado.
- Antes de executar, percorra o código de cima para baixo e anote o estado das principais variáveis.
- Execute e compare a saída com sua previsão.
- Altere um único valor ou operação e observe o efeito; não modifique muitas coisas ao mesmo tempo.
- Nos exemplos com banco, identifique quais linhas são apenas SELECT e quais realmente executam INSERT, UPDATE ou DELETE.
- Quando uma estrutura é alterada, pergunte explicitamente: mudou somente a memória ou também houve SQL de persistência?

Como resolver uma atividade

As atividades foram desenhadas para repetir operações essenciais, mas também para provocar comparação, diagnóstico e escolha. Em vez de procurar uma frase pronta, transforme o enunciado em decisões.

Pergunta	O que ela revela
Quantos elementos/registros preciso tratar?	um registro pode sugerir tuple; vários normalmente chegam como list
A ordem importa?	list/deque/pilha preservam políticas de ordem; set não representa posição
Duplicatas têm significado?	se não, set pode ser apropriado
Existe uma chave natural de acesso?	dict pode criar índice temporário
Quem deve sair primeiro?	LIFO → pilha; FIFO → deque; prioridade → heap
Há hierarquia?	árvore/recursividade
Há conexões entre várias unidades?	grafo; BFS/DFS conforme a pergunta
A alteração deve sobreviver a novo SELECT?	é necessário SQL de persistência, não apenas mutação Python

O banco didático lojas_rede

O banco representa uma rede varejista fictícia simplificada. Ele contém apenas entidades necessárias para dar contexto aos exemplos: clientes, fornecedores, categorias, produtos, lojas/CDs, estoque por unidade, canais de venda, pedidos, itens, histórico de status, solicitações de reposição, etapas do processo e rotas logísticas. O objetivo não é reproduzir um ERP completo.

Tabela	Papel nos exemplos
clientes	registros, tuples, dicts, busca e hashing por CPF/CNPJ
fornecedores	relacionamento simples de produtos
categorias	hierarquia, recursividade, árvores e DFS

Tabela	Papel nos exemplos
produtos	listas, busca, ordenação, hashing e BST por preço
unidades	lojas e centros de distribuição
estoques	disponibilidade por unidade, sets e transações
canais_venda	agrupamentos e contagens
pedidos / itens_pedido	CRUD, agregações e transações
historico_status_pedido	pilha e auditoria temporal
solicitacoes_reposicao	fila FIFO e heap de prioridade
etapas_reposicao	lista encadeada e recursividade
rotas	grafos, BFS e DFS

Conexão e cursor

Os exemplos importam `criar_conexao()` do módulo `database`. A conexão representa a sessão com o MySQL; o cursor envia comandos e recupera resultados. O cursor padrão é importante didaticamente porque `fetchone()` devolve uma tuple e `fetchall()` devolve uma list de tuples.

Padrão mínimo de consulta

```
from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

cursor.execute(
    "SELECT id_cliente, nome, email FROM clientes WHERE id_cliente = %s",
    (1,)
)
cliente = cursor.fetchone()

cursor.close()
conexao.close()
```

MEMÓRIA X PERSISTÊNCIA — cliente é uma tuple na memória. A variável não é uma “linha conectada” ao banco. Mesmo que você crie outra estrutura com valores diferentes, o MySQL continua igual até receber um comando de persistência.

Mapa rápido dos dados que aparecem nas atividades

ID	Unidade	Tipo	Cidade/UF
1	Sete Lagoas Centro	LOJA	Sete Lagoas/MG
2	Belo Horizonte Centro	LOJA	Belo Horizonte/MG
3	Contagem Eldorado	LOJA	Contagem/MG
4	Betim Centro	LOJA	Betim/MG
5	Divinópolis Centro	LOJA	Divinópolis/MG
6	Montes Claros Centro	LOJA	Montes Claros/MG
7	Uberlândia Centro	LOJA	Uberlândia/MG
8	Juiz de Fora Centro	LOJA	Juiz de Fora/MG
9	Sao Paulo Centro	LOJA	Sao Paulo/SP
10	Brasília Centro	LOJA	Brasília/DF
20	CD Contagem	CD	Contagem/MG
21	CD Uberlândia	CD	Uberlândia/MG

ID	Canal de venda
1	Loja física
2	Site
3	Aplicativo

ID	Canal de venda
4	Marketplace
5	WhatsApp

ID	Categoria	Pai
1	Cabelos	-
2	Shampoo	1
3	Condicionador	1
4	Tratamento Capilar	1
5	Perfumaria	-
6	Perfumes	5
7	Maquiagem	-
8	Rosto	7
9	Labios	7
10	Higiene	-
11	Higiene Bucal	10
12	Higiene Corporal	10
13	Cuidados com a Pele	-
14	Facial	13
15	Corporal	13
16	Acessorios	-

Mapa da sequência

#	Pasta	Foco
01	01_tuplas	Tuplas
02	02_listas	Listas
03	03_dicionarios	Dicionários
04	04_sets	Sets
05	05_try_except	Tratamento de exceções
06	06_funcoes_parametro_retorno	Funções aplicadas a coleções, consultas e efeitos colaterais
07	07_recursos_iteracao_transformacao	Recursos de iteração e transformação
08	08_crud_persistencia	CRUD e persistência
09	09_transacoes	Transações
10	10_complexidade	Complexidade
11	11_pilhas	Pilhas
12	12_filas_deque	Filas e deque
13	13_busca_linear	Busca linear
14	14_busca_binaria	Busca binária e bisect
15	15_ordenacao	Ordenação
16	16_rekursividade	Recursividade
17	17_lista_encadeada	Lista encadeada
18	18_hashing	Hashing
19	19_arvores	Árvores
20	20_heap_prioridade	Heap e fila de prioridade
21	21_grafos	Grafos
22	22_bfs	BFS
23	23_dfs	DFS
24	24_integracao	Integração

1. Tuplas: registros posicionais e resultados de uma consulta

A primeira estrutura estudada é a tuple porque ela permite fazer uma transição direta entre o modelo relacional e os dados que chegam à memória do programa. Uma linha consultada pelo cursor padrão do MySQL Connector costuma ser materializada como uma tupla: os valores aparecem em uma ordem definida pelo SELECT e são acessados por posição. Isso cria uma oportunidade importante para separar duas ideias desde o início: a linha persistida no banco e a estrutura Python que contém uma cópia dos valores recuperados naquele momento.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Criação, posição e imutabilidade

Uma tupla é uma sequência ordenada. Isso significa que cada elemento ocupa uma posição e pode ser acessado por índice. O primeiro índice é 0; índices negativos contam a partir do final. Diferentemente da list, a tuple não permite substituir, inserir ou remover elementos depois de criada. A imutabilidade é útil quando o conjunto de valores deve ser tratado como um registro estável durante aquele processamento. Ela não significa, porém, que os dados do banco se tornaram imutáveis: o banco continua podendo ser alterado por um UPDATE explícito.

Operações de consulta

Como a tupla é uma sequência, len() informa a quantidade de elementos, in e not in testam pertencimento, count() conta ocorrências e index() devolve a posição da primeira ocorrência. Esses recursos não alteram a tupla. index() gera ValueError se o valor não existir, por isso ele só deve ser usado quando a ausência tiver sido previamente tratada ou quando essa exceção fizer sentido.

Desempacotamento

O desempacotamento associa posições a nomes de variáveis e melhora a legibilidade quando a estrutura da tupla é conhecida. Em vez de espalhar produto[0], produto[1] e produto[2] pelo programa, é possível escrever id_produto, sku, nome = produto. O número de variáveis deve coincidir com o número de elementos, salvo quando se utiliza * para capturar uma parte variável. O mesmo mecanismo explica retornos múltiplos de funções: return a, b produz uma tupla que pode ser desempacotada.

Tupla com um elemento

Uma tupla de um único elemento precisa da vírgula: (10,) é tuple; (10) é apenas o inteiro 10 entre parênteses. Esse detalhe aparece também nos parâmetros do conector: quando a consulta possui apenas um parâmetro, (id_cliente,) representa a tupla de parâmetros corretamente.

Quando usar

- representar um pequeno registro cuja ordem dos campos é conhecida
- receber uma linha de fetchone() com o cursor padrão
- retornar múltiplos valores de uma função
- usar uma sequência como chave de dicionário quando todos os seus elementos forem hashable

Evite quando

- quando os campos precisam ser atualizados individualmente muitas vezes
- quando nomes de campo são mais importantes que posições; nesse caso um dict pode comunicar melhor a intenção
- quando o conjunto de dados cresce dinamicamente com inserções e remoções

Erros e armadilhas comuns

- confundir (valor) com (valor,)
- presumir que alterar ou reconstruir uma tupla altera a linha no MySQL
- acessar uma posição que não existe e provocar IndexError
- usar index() sem considerar que o valor pode não existir

No banco lojas_rede

Nos exemplos 06 e 07, fetchone() é o elo central. Um SELECT como SELECT id_cliente, nome, email FROM clientes WHERE id_cliente = %s define a ordem dos campos. Se houver resultado, fetchone() entrega uma tuple; se não houver linha correspondente, entrega None. Por isso o programa deve testar o resultado antes de acessar

índices. Criar outra tupla com valores diferentes modifica apenas a memória. A persistência exige UPDATE e, em uma operação transacional, commit().

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como append(), pop(), add(), alteração de dict, popleft(), heappop() ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_criacao_e_tipo.py	criacao e tipo
02_indices_e_indices_negativos.py	índices e índices negativos
03_len_in_count_index.py	len in count index
04_desempacotamento.py	desempacotamento
05_tupla_um_elemento_e_aninhamento.py	tupla um elemento e aninhamento
06_fetchone_retorna_tupla.py	fetchone retorna tupla
07_fetchone_none_e_memoria_nao_e_banco.py	fetchone none e memoria nao e banco

Trecho para leitura — exemplos/01_tuplas/01_criacao_e_tipo.py

```
cliente = (1, "Ana Martins", "ana.martins@email.com")
print(cliente)
print(type(cliente))
```

Trecho para leitura — exemplos/01_tuplas/06_fetchone_retorna_tupla.py

```
from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

cursor.execute("""
    SELECT id_cliente, nome, email, cidade, uf
    FROM clientes
    WHERE id_cliente = %s
""", (1,))
cliente = cursor.fetchone()
print(cliente)
print(type(cliente))

cursor.close()
conexao.close()
```

Como pensar nas atividades

- identifique quantos valores existem e em que ordem
- pergunte se a estrutura precisa mudar de tamanho ou ter campos alterados
- ao receber fetchone(), trate primeiro a possibilidade de None
- se a atividade pede persistência, procure uma operação SQL; apenas reconstruir a tuple não basta

Arquivo	Foco	Tipo
01_criar_tupla.py	Criar e inspecionar uma tupla	fixacao
02_indices.py	Acesso por índice	fixacao
03_desempacotar.py	Desempacotamento	fixacao
04_count_index.py	count() e index()	fixacao
05_fetchone_cliente.py	fetchone() e tuple	aplicacao com banco
06_fetchone_produto.py	Acessar campos da tupla do banco	aplicacao com banco
07_tupla_um_item.py	Tupla com um elemento	diagnostico
08_quando_tupla.py	Quando a tupla é adequada?	escolha de recurso
09_nova_tupla.py	Nova tupla não é UPDATE	memoria x persistencia
10_imutabilidade.py	Tentar alterar uma tupla	diagnostico
11_none.py	fetchone() pode retornar None	aplicacao com banco

Arquivo	Foco	Tipo
12_retorno_multiplo.py	Tupla como retorno múltiplo	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
tuple literal	registro = (1, 'Ana', 'MG')
índice	registro[0] / registro[-1]
tamanho	len(registro)
pertencimento	'Ana' in registro
contagem	registro.count(valor)
posição	registro.index(valor)
desempacotar	id_cliente, nome, uf = registro
um elemento	(id_cliente,)

2. Listas: coleções mutáveis, fatiamento e resultados de fetchall()

Uma consulta que retorna várias linhas cria um problema diferente: não há apenas um registro, mas uma coleção de registros. O `fetchall()` do cursor padrão fornece uma list cujos elementos normalmente são tuples. A lista é mutável, preserva ordem, aceita repetições e oferece operações de inclusão, remoção, substituição, ordenação e fatiamento. Justamente por ser mutável, ela também é o primeiro lugar em que a diferença entre “mudei a coleção em memória” e “mudei o banco” precisa ser observada de forma concreta.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Inserir: `append()`, `insert()` e `extend()`

`append(x)` acrescenta um único elemento ao final. Se `x` for outra lista, essa lista inteira vira um único elemento. `insert(i, x)` insere um elemento em uma posição, deslocando os posteriores. `extend(iterável)` percorre o iterável recebido e acrescenta seus elementos individualmente. A escolha depende da intenção: adicionar um registro, inserir em posição específica ou concatenar coleções.

Remover: `remove()`, `pop()` e `clear()`

`remove(valor)` procura o primeiro valor igual e o remove; se não encontrar, gera `ValueError`. `pop(indice)` remove pela posição e devolve o elemento retirado, sendo `pop()` sem argumento equivalente a retirar o último. `clear()` esvazia a lista. Nenhuma dessas operações executa DELETE no banco; elas atuam somente sobre a list existente no processo Python.

Fatiamento

A forma `lista[início:fim:passo]` produz uma nova lista com parte dos elementos. O limite final é exclusivo. Omitir início começa no primeiro elemento; omitir fim vai até o final; passo negativo permite percursos invertidos, como `lista[::-1]`. Slicing é útil para páginas, amostras, primeiros/últimos itens e cópias rasas simples.

`sort()`, `sorted()` e ordem

`list.sort()` altera a própria lista e retorna `None`. `sorted(iterável)` cria uma nova lista ordenada e preserva a coleção original. Ambos aceitam `key` e `reverse`, que serão aprofundados posteriormente. Para dados que já estão no MySQL, também existe a possibilidade de pedir ORDER BY ao banco, evitando trazer dados em uma ordem para depois reordená-los sem necessidade.

Referência, cópia e cópia rasa

`b = a` não copia uma lista: os dois nomes passam a referenciar o mesmo objeto. `a.copy()`, `list(a)` e `a[:]` criam uma nova lista, mas a cópia é rasa; objetos mutáveis internos continuam compartilhados. Quando há estruturas aninhadas e é necessário duplicar também os objetos internos, `copy.deepcopy()` é uma ferramenta útil. A atividade deve sempre perguntar: quero dois nomes para o mesmo objeto ou coleções independentes?

Quando usar

- coleções ordenadas que podem crescer, diminuir ou ser alteradas
- resultados de `fetchall()`
- percursos sequenciais em que duplicatas são permitidas
- filtragem, ordenação e fatiamento de dados já carregados em memória

Evite quando

- quando a principal necessidade é unicidade e pertencimento rápido; `set` pode ser melhor
- quando a principal necessidade é acesso direto por chave; `dict` pode ser melhor
- como fila de grande volume com remoção frequente no início; `deque` é mais apropriado

Erros e armadilhas comuns

- escrever `lista = lista.append(x)` e terminar com `None`
- confundir `remove(valor)` com `pop(indice)`
- acreditar que `append()` equivale a `INSERT` ou `pop()` a `DELETE`
- confundir cópia com nova referência ao mesmo objeto
- usar `sort()` quando precisava preservar a ordem original

No banco lojas_rede

Os exemplos 08 a 10 usam produtos e estoques. `fetchall()` retorna uma list de tuples. É possível filtrar os registros com `for/if`, ordenar em memória ou acrescentar uma tuple fictícia com `append()`. Esse acréscimo só existe até a lista ser descartada. Um novo `SELECT` reconstrói a coleção a partir do estado persistido e o item fictício desaparece porque nunca houve `INSERT`.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de `SELECT` é local ao processo Python. Operações como `append()`, `pop()`, `add()`, alteração de `dict`, `popleft()`, `heappop()` ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_criacao_acesso_alteracao.py	criacao acesso alteracao
02_append_insert_extend.py	append insert extend
03_remove_pop_clear.py	remove pop clear
04_len_in_count_index.py	len in count index
05_slicing.py	slicing
06_sort_sorted_reverse.py	sort sorted reverse
07_referencia_copy_e_copia_rasa.py	referencia copy e copia rasa
08_fetchall_lista_de_tuplas.py	fetchall lista de tuplas
09_filtrar_dados_do_banco_em_memoria.py	filtrar dados do banco em memoria
10_append_nao_e_insert.py	append nao e insert

Trecho para leitura — exemplos/02_listas/02_append_insert_extend.py

```
produtos = ["Shampoo Nutritivo 350 ml", "Condicionador Nutritivo 350 ml"]
produtos.append("Perfume Floral 100 ml")
produtos.insert(1, "Mascara Capilar 300 g")
produtos.extend(["Protetor Solar Facial FPS 60", "Hidratante Corporal 400 ml"])
print(produtos)
```


Trecho para leitura — exemplos/02_listas/07_referencia_copy_e_copia_rasa.py

```

a = ["Shampoo", "Condicionador"]
b = a
b.append("Perfume")
print("a:", a)
print("b:", b)

c = a.copy()
c.append("Protetor Solar")
print("a após cópia:", a)
print("c:", c)

matriz = [[1, 2], [3, 4]]
copia = matriz.copy()
copia[0][0] = 999
print("Cópia rasa afeta lista interna compartilhada:", matriz)

```

Trecho para leitura — exemplos/02_listas/08_fetchall_lista_de_tuplas.py

```

from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

cursor.execute("""
    SELECT id_produto, sku, nome, preco
    FROM produtos
    ORDER BY id_produto
    LIMIT 5
""")
produtos = cursor.fetchall()
print(type(produtos))
print(type(produtos[0]))
for produto in produtos:
    print(produto)

cursor.close()
conexao.close()

```

Como pensar nas atividades

- defina se a ordem importa e se duplicatas são aceitáveis
- decida se a operação modifica a coleção ou deve produzir outra
- diferencie valor e índice antes de escolher remove() ou pop()
- se o dado veio do banco, pergunte se a atividade quer apenas processamento em memória ou persistência
- antes de copiar, identifique se há listas/dicts aninhados que exigem cópia profunda

Arquivo	Foco	Tipo
01_append_insert.py	append() e insert()	fixacao
02_extend.py	append() x extend()	fixacao
03_remove_pop.py	remove() x pop()	fixacao
04_slicing.py	Fatiamento	fixacao
05_sort_sorted.py	sort() x sorted()	fixacao
06_referencia.py	Referência compartilhada	conceito
07_copy.py	Cópia rasa	conceito
08_fetchall.py	fetchall() e list de tuples	aplicacao com banco
09_filtro.py	Filtrar em memória	aplicacao com banco
10_ordenar.py	Ordenar registros em memória	aplicacao com banco
11_append_nao_insert.py	append() não é INSERT	memoria x persistencia
12_pop_nao_delete.py	pop() não é DELETE	memoria x persistencia

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
adicionar um	<code>lista.append(x)</code>
inserir	<code>lista.insert(i, x)</code>
adicionar vários	<code>lista.extend(itens)</code>
remover valor	<code>lista.remove(x)</code>
remover posição	<code>x = lista.pop(i)</code>
fatiar	<code>lista[inicio:fim:passo]</code>
ordenar no lugar	<code>lista.sort()</code>
nova ordenada	<code>sorted(lista)</code>
cópia rasa	<code>lista.copy()</code>
buscar posição	<code>lista.index(x)</code>

3. Dicionários: chaves, registros nomeados e índices em memória

Tuplas são compactas, mas obrigam o programador a lembrar que a posição 0 é um ID, a 1 é um SKU e assim por diante. Um dict troca a ideia de posição pela associação chave → valor. Essa forma é especialmente útil para representar registros com campos nomeados e para construir índices temporários em memória, como sku → registro do produto. O dicionário é mutável, mas sua mutabilidade continua não significando sincronização com o banco.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Acesso direto e get()

`d[chave]` devolve o valor associado e gera `KeyError` se a chave não existir. `d.get(chave)` devolve `None` por padrão, ou um valor padrão informado como segundo argumento. `get()` é adequado quando a ausência é esperada; acesso com colchetes é adequado quando a ausência representa realmente um erro de programação ou de dados.

keys(), values() e items()

`keys()` oferece as chaves, `values()` os valores e `items()` os pares chave/valor. No for, percorrer diretamente o dicionário percorre as chaves. `items()` é a forma mais clara quando chave e valor são necessários ao mesmo tempo. Essas visões acompanham o dicionário; se ele mudar, a visão reflete a mudança.

update(), pop() e.setdefault()

`update()` incorpora pares de outro mapeamento ou argumentos nomeados e substitui valores de chaves existentes. `pop(chave)` remove a chave e devolve o valor. `setdefault(chave, padrão)` devolve o valor existente ou cria a chave com o padrão. `setdefault()` é útil em agrupamentos simples, embora `defaultdict` possa ser mais expressivo quando o padrão é recorrente.

Tupla para dict com zip()

Quando um registro chega como tuple e existe uma sequência de nomes de colunas na mesma ordem, `dict(zip(colunas, registro))` cria uma representação nomeada. `zip()` forma pares até o menor dos iteráveis, portanto listas de tamanhos diferentes podem ocultar campos; o programador deve garantir que colunas e valores correspondam.

Dicionário como índice temporário

Após um `fetchall()`, criar `{produto[1]: produto for produto in produtos}` transforma uma coleção sequencial em um índice por SKU. O custo de construção é linear, mas buscas posteriores por chave têm custo médio constante. Isso

faz sentido quando haverá várias buscas no mesmo conjunto carregado. Para uma consulta pontual, `SELECT ... WHERE sku = %s` pode ser mais simples e evita trazer dados desnecessários.

Quando usar

- representar registros com campos nomeados
- agrupar ou contar por chave
- criar índices temporários, por exemplo SKU → produto
- precisar de acesso médio rápido por uma chave hashable

Evite quando

- quando a ordem posicional compacta da tuple já comunica bem o registro
- quando os dados são apenas uma sequência sem chave natural
- quando a coleção precisa permitir várias entradas com a mesma chave sem agrupamento explícito

Erros e armadilhas comuns

- acessar chave inexistente com colchetes quando a ausência é normal
- supor que alterar `dict['preco']` atualiza `produtos.preco`
- usar uma chave mutável, como `list`
- esquecer que chaves repetidas substituem o valor anterior

No banco lojas_rede

Os exemplos 07 a 09 convertem registros reais de produtos para dicionários e criam um índice por SKU. Essa representação melhora a legibilidade em Python, porém é uma cópia dos valores consultados. Alterar `produto['preco']` muda apenas o objeto. Para persistir, é necessário executar `UPDATE produtos SET preco = %s WHERE id_produto = %s` e confirmar a transação quando apropriado.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de `SELECT` é local ao processo Python. Operações como `append()`, `pop()`, `add()`, alteração de `dict`, `popleft()`, `heappop()` ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_criacao_acesso_alteracao.py	criacao acesso alteracao
02_get_e_valor_padrao.py	get e valor padrao
03_keys_values_items.py	keys values items
04_update_pop_setdefault.py	update pop setdefault
05_lista_de_dicionarios.py	lista de dicionarios
06_tupla_para_dict_com_zip.py	tupla para dict com zip
07_registro_banco_para_dicionario.py	registro banco para dicionario
08_dict_mutavel_nao_atualiza_banco.py	dict mutavel nao atualiza banco
09_indexar_produtos_por_sku.py	indexar produtos por sku

Trecho para leitura — exemplos/03_dicionarios/02_get_e_valor_padrao.py

```
cliente = {"id": 1, "nome": "Ana Martins", "cidade": "Sete Lagoas"}
print(cliente.get("cidade"))
print(cliente.get("telefone"))
print(cliente.get("telefone", "não informado"))
```

Trecho para leitura — exemplos/03_dicionarios/06_tupla_para_dict_com_zip.py

```
colunas = ("id_cliente", "nome", "cidade", "uf")
registro = (1, "Ana Martins", "Sete Lagoas", "MG")
cliente = dict(zip(colunas, registro))
print(cliente)
```

Trecho para leitura — exemplos/03_dicionarios/09_indexar_produtos_por_sku.py

```

from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

cursor.execute("SELECT sku, nome, preco FROM produtos")
registros = cursor.fetchall()
por_sku = {}
for sku, nome, preco in registros:
    por_sku[sku] = {"nome": nome, "preco": preco}
print(por_sku["PERF-FEM-001"])

cursor.close()
conexao.close()

```

Como pensar nas atividades

- identifique qual dado funciona naturalmente como chave
- pergunte se a ausência da chave é erro ou situação esperada
- se converter tuple para dict, confirme a ordem das colunas
- se criar índice em memória, avalie se haverá buscas repetidas suficientes para justificar a estrutura

Arquivo	Foco	Tipo
01_criar_acessar.py	Criar e acessar dicionário	fixacao
02_alterar_adicionar.py	Alterar e adicionar chaves	fixacao
03_get.py	Acesso seguro com get()	fixacao
04_items.py	Percorrer items()	fixacao
05_update_pop.py	update(), setdefault() e pop()	fixacao
06_lista_dicts.py	Lista de dicionários	aplicacao
07_zip.py	tuple para dict	transformacao
08_registro_dict.py	Converter registro do banco	aplicacao com banco
09_dict_nao_update.py	Alterar dict não é UPDATE	memoria x persistencia
10_dict_vs_tuple.py	Quando usar dict em vez de tuple?	escolha de recurso
11_indice_sku.py	Dicionário como índice	aplicacao com banco
12_keyerror_get.py	KeyError x get()	diagnostico

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
acessar	d['nome']
acesso opcional	d.get('telefone')
pares	d.items()
alterar/adicionar	d['preco'] = 10
mesclar	d.update(outro)
remover	valor = d.pop('chave')
padrão	d.setdefault('itens', [])
tuple → dict	dict(zip(colunas, registro))

4. Sets: unicidade, pertencimento e comparação entre grupos

Um set representa um conjunto de valores únicos. Ele não é uma sequência indexada: a pergunta principal deixa de ser “qual elemento está na posição 3?” e passa a ser “este valor pertence ao conjunto?”. No cenário lojas_rede,

conjuntos são especialmente úteis para representar produtos disponíveis em uma unidade, estados presentes na rede ou canais distintos utilizados em um recorte de pedidos.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Unicidade e criação

set(iterável) elimina duplicatas. O literal {1, 2, 3} cria um conjunto, mas {} cria um dict vazio; para conjunto vazio use set(). A ordem de iteração de um set não deve ser usada como parte da lógica, pois a estrutura não promete uma ordem posicional estável.

add(), update(), remove() e discard()

add(x) insere um elemento. update(iterável) insere os elementos de um ou mais iteráveis. remove(x) gera KeyError se o item não existir; discard(x) simplesmente não faz nada nesse caso. A escolha entre remove e discard expressa se a ausência é um erro ou uma situação aceitável.

Operações de conjuntos

A união A | B reúne todos os elementos; a interseção A & B mantém apenas os comuns; A - B mantém os que estão em A e não em B; A ^ B mantém os exclusivos de um dos dois. issubset(), issuperset() e isdisjoint() respondem perguntas de relação entre conjuntos.

Disponibilidade por unidade

Ao consultar estoques com quantidade > 0 e transformar SKUs em sets, a interseção responde quais produtos estão disponíveis simultaneamente em duas unidades, enquanto as diferenças mostram exclusividades. Essa é uma escolha semântica: a estrutura representa exatamente a ideia matemática de conjuntos de produtos.

Quando usar

- eliminar duplicatas
- testar pertencimento com frequência
- comparar grupos por união/interseção/diferença
- representar conjuntos de IDs, SKUs, UFs ou canais únicos

Evite quando

- quando a ordem ou a posição precisa ser preservada
- quando valores repetidos carregam significado
- quando é necessário associar cada chave a dados adicionais; use dict

Erros e armadilhas comuns

- tentar usar set[0]
- esperar uma ordem fixa de exibição
- usar remove() sem considerar KeyError
- acreditar que add() insere produto/estoque no MySQL

No banco lojas_rede

Nos exemplos 05 a 07, o set é construído a partir de SELECT sobre estoques e produtos. Adicionar um SKU ao conjunto em memória não cria estoque e não modifica quantidade. Um novo SELECT continua refletindo exclusivamente o banco. Essa distinção é particularmente importante porque a operação add() soa como “adicionar produto”, mas o seu efeito é apenas sobre o objeto Python.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como append(), pop(), add(), alteração de dict, popleft(), heappop() ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_criacao_e Eliminacao_duplicatas.py	criacao e eliminacao duplicatas
02_add_update_remove_discard.py	add update remove discard

Arquivo	O que observar
03_uniao_intersecao_diferenca.py	uniao intersecao diferenca
04_subset_superset_disjoint.py	subset superset disjoint
05_produtos_disponiveis_em_unidade.py	produtos disponiveis em unidade
06_comparar_disponibilidade_duas_unidades.py	comparar disponibilidade duas unidades
07_set_em_memoria_nao_muda_estoque.py	set em memoria nao muda estoque

Trecho para leitura — exemplos/04_sets/03_uniao_intersecao_diferenca.py

```
canais_cliente_a = {"Loja fisica", "Site", "WhatsApp"}
canais_cliente_b = {"Site", "Aplicativo"}

print(canais_cliente_a | canais_cliente_b) # união
print(canais_cliente_a & canais_cliente_b) # interseção
print(canais_cliente_a - canais_cliente_b) # diferença
print(canais_cliente_a ^ canais_cliente_b) # diferença simétrica
```

Trecho para leitura — exemplos/04_sets/06_comparar_disponibilidade_duas_unidades.py

```
from database import criar_conexao

conexao = criar_conexao(); cursor = conexao.cursor()
def skus_da_unidade(id_unidade):
    cursor.execute("""
        SELECT p.sku FROM estoques e
        JOIN produtos p ON p.id_produto=e.id_produto
        WHERE e.id_unidade=%s AND e.quantidade > 0
        """, (id_unidade,))
    return {linha[0] for linha in cursor.fetchall()}

sete_lagoas = skus_da_unidade(1)
bh = skus_da_unidade(2)
print("Em ambas:", sete_lagoas & bh)
print("Somente Sete Lagoas:", sete_lagoas - bh)
print("Em pelo menos uma:", sete_lagoas | bh)
cursor.close(); conexao.close()
```

Como pensar nas atividades

- se a atividade fala em elementos únicos, comuns ou exclusivos, considere set
- se a ordem de chegada/importância importa, set provavelmente não é a melhor escolha
- diferencie remover de uma coleção em memória de alterar o estoque persistido
- em comparações entre unidades, decida qual conjunto representa cada lado antes da operação

Arquivo	Foco	Tipo
01_duplicatas.py	Eliminar duplicatas	fixacao
02_add_update.py	add() e update()	fixacao
03_remove_discard.py	remove() x discard()	fixacao
04_operacoes.py	Operações de conjuntos	fixacao
05_subset.py	Subconjunto e disjunção	fixacao
06_produtos_unidade.py	PRODUTOS DISPONÍVEIS EM UMA UNIDADE	aplicação com banco
07_intersecao_unidades.py	INTERSEÇÃO DE DISPONIBILIDADE	aplicação com banco
08_diferenca_unidades.py	DIFERENÇA ENTRE UNIDADES	aplicação com banco
09_set_ou_list.py	List ou set?	escolha de recurso
10_add_nao_insert.py	ADD NÃO PERSISTE	memória x persistência
11_sem_indice.py	Set não é sequência indexada	diagnostico
12_clientes_cidades.py	Conjuntos a partir de clientes	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
vazio	<code>s = set()</code>
adicionar	<code>s.add(x)</code>
vários	<code>s.update(itens)</code>
remover estrito	<code>s.remove(x)</code>
remover tolerante	<code>s.discard(x)</code>
união	<code>a b</code>
interseção	<code>a & b</code>
diferença	<code>a - b</code>
simétrica	<code>a ^ b</code>
subconjunto	<code>a.issubset(b)</code>

5. Tratamento de exceções: falhas esperadas, diagnóstico e limpeza de recursos

Estruturas de dados e banco introduzem novas formas de falha: converter uma entrada inválida, acessar índice inexistente, usar uma chave ausente ou violar restrições do MySQL. `try/except` não serve para esconder erros; serve para reconhecer situações excepcionais que o programa sabe tratar. Um bom tratamento é específico, preserva informações úteis para diagnóstico e garante que recursos como cursor e conexão sejam fechados.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

`try` e `except` específicos

O bloco `try` deve conter apenas a parte que pode gerar a exceção que está sendo tratada. `except ValueError`, `IndexError` ou `KeyError` comunica exatamente qual situação foi prevista. Capturar `Exception` cedo demais pode esconder defeitos e dificultar o diagnóstico. Quando vários tipos recebem o mesmo tratamento, `except (ValueError, TypeError):` é válido.

`else` e `finally`

`else` executa apenas quando o `try` termina sem exceção e é útil para separar o caminho de sucesso. `finally` executa independentemente do resultado e é apropriado para limpeza, como fechar cursor/conexão. Ele não deve ser usado para mascarar o fluxo com `returns` ou `raises` desnecessários.

Exceções do MySQL

Restrições `UNIQUE` e `FOREIGN KEY` podem gerar `mysql.connector.Error`. Em operações que modificam dados, o tratamento geralmente precisa de `rollback()` para abandonar alterações não confirmadas. O objeto da exceção possui informações úteis; mostrá-las durante o aprendizado ajuda a entender a causa em vez de imprimir apenas “deu erro”.

None não é exceção

`fetchone()` retornar `None` quando nenhum registro corresponde é um resultado normal da consulta, não uma exceção. O programa deve testá-lo com `if registro is None`. Usar `try/except` para substituir uma simples verificação de ausência torna a lógica menos clara.

Quando usar

- entrada do usuário pode ter formato inválido
- operações de coleção têm condições excepcionais previsíveis
- restrições de banco podem falhar
- é necessário garantir limpeza/rollback

Evite quando

- envolver o programa inteiro em `try/except Exception`
- usar exceções para controlar situações normais como `fetchone() == None`
- capturar e ignorar o erro sem ação nem diagnóstico

Erros e armadilhas comuns

- colocar except genérico antes do específico
- esquecer rollback() após falha em uma transação
- fechar cursor/conexão apenas no caminho de sucesso
- confundir KeyError com get() retornando None

No banco_lojas_rede

Os exemplos 06 e 07 usam erros reais do MySQL e finally. A ideia central é preservar a integridade: se um INSERT viola UNIQUE, a exceção deve ser tratada e a transação pode ser revertida. Depois, cursor e conexão devem ser fechados mesmo que algo tenha falhado.

Como ler os exemplos da pasta

Arquivo	O que observar
01_value_error.py	value error
02_index_error.py	index error
03_key_error_e_get.py	key error e get
04_multiplos_except.py	multiplos except
05_else_finally.py	else finally
06_erro_mysql_unique.py	erro mysql unique
07_finally_fecha_recursos.py	finally fecha recursos
08_captura_especifica_antes_de_generica.py	captura especifica antes de generica

Trecho para leitura — exemplos/05_try_except/05_else_finally.py

```
texto = "25"
try:
    numero = int(texto)
except ValueError:
    print("Conversão falhou.")
else:
    print("Conversão concluída:", numero)
finally:
    print("Este bloco sempre é executado.")
```

Trecho para leitura — exemplos/05_try_except/06_erro_mysql_unique.py

```
import mysql.connector
from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()
try:
    cursor.execute("""
        INSERT INTO clientes (nome, cpf_cnpj, email)
        VALUES (%s, %s, %s)
        """, ("Duplicado", "111.111.111-01", "dup@email.com"))
    conexao.commit()
except mysql.connector.Error as erro:
    conexao.rollback()
    print("Erro do MySQL:", erro)
finally:
    cursor.close()
    conexao.close()
```

Como pensar nas atividades

- descubra qual operação pode falhar e qual exceção ela produz
- pergunte se a situação é realmente excepcional ou apenas ausência esperada
- decida se a falha precisa de rollback
- garanta que a limpeza ocorra em todos os caminhos

Arquivo	Foco	Tipo
---------	------	------

Arquivo	Foco	Tipo
01_valueerror.py	Tratar ValueError	fixacao
02_indexerror.py	Tratar IndexError	fixacao
03_keyerror.py	Tratar KeyError	fixacao
04_multiplos.py	Múltiplos except	fixacao
05_else_finally.py	else e finally	fixacao
06_get_ou_except.py	get() ou KeyError?	escolha de recurso
07_unique.py	Erro de UNIQUE no MySQL	aplicacao com banco
08_foreign_key.py	Erro de chave estrangeira	aplicacao com banco
09_finally.py	Fechar cursor e conexão	aplicacao com banco
10_exception_generica.py	Captura genérica demais	diagnostico
11_validar_id.py	Entrada e consulta segura	integração
12_none_nao_excecao.py	None não é exceção	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
estrutura	try: ... / except TipoErro: ...
vários tipos	except (ValueError, TypeError):
sucesso	else:
sempre	finally:
erro do MySQL	except mysql.connector.Error as erro:
ausência normal	if registro is None:

6. Funções aplicadas a coleções, consultas e efeitos colaterais

Funções já não são um conteúdo isolado: agora elas organizam operações sobre coleções e acesso ao banco. Uma boa função deixa claro o que recebe, o que devolve e se modifica objetos externos. Em estruturas mutáveis, esse último ponto é essencial: passar uma list ou dict permite que a função altere o mesmo objeto visto por quem chamou.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Parâmetros com objetos mutáveis

Quando uma list é passada como argumento, a função recebe uma referência ao mesmo objeto. `append()`, `pop()` ou atribuições dentro da função podem produzir efeitos visíveis fora dela. Isso pode ser desejado, mas deve ser deliberado. Se a intenção é produzir uma nova coleção, crie e retorne outra lista em vez de modificar silenciosamente a recebida.

Valor padrão mutável

Parâmetros padrão são avaliados quando a função é definida, não a cada chamada. Por isso `def registrar(item, itens=[]):` reutiliza a mesma lista entre chamadas. O padrão seguro é `itens=None` e criar `[]` dentro da função quando necessário.

*args e **kwargs

*args reúne argumentos posicionais extras em uma tuple; **kwargs reúne argumentos nomeados extras em um dict. Esses recursos são úteis em funções flexíveis e também permitem desempacotar estruturas na chamada: `funcao(*tupla)` e `funcao(**dicionario)`. Flexibilidade excessiva, porém, pode esconder contratos; prefira parâmetros explícitos quando o formato é conhecido.

Funções de acesso ao banco

Uma função como `buscar_cliente(cursor, id_cliente)` pode executar `SELECT` e retornar uma tuple ou `None`. Outra pode retornar uma list de produtos. Separar consulta, transformação e apresentação permite testar e reutilizar cada parte sem misturar `input()`, `print()` e SQL em um único bloco.

Quando usar

- encapsular uma operação que se repete
- separar consulta, transformação e apresentação
- dar nomes a regras como `produtos_estoque_baixo`
- retornar coleções em vez de imprimir dentro da função

Evite quando

- funções que fazem muitas tarefas desconectadas
- efeitos colaterais inesperados em listas/dicts recebidos
- parâmetros mutáveis como valores padrão
- usar `*args/**kwargs` quando parâmetros explícitos seriam mais claros

Erros e armadilhas comuns

- confundir `print` com `return`
- alterar a lista recebida sem perceber
- reutilizar lista padrão entre chamadas
- retornar `cursor/ conexão` ainda necessários após fechá-los

No banco lojas_rede

Os exemplos 06 a 08 mostram funções que recebem conexão/cursor ou parâmetros de consulta e devolvem dados em memória. A função não cria sincronização automática: depois de retornar uma list, alterações nessa lista continuam locais até que outra função execute SQL de persistência.

Como ler os exemplos da pasta

Arquivo	O que observar
01_receber_tupla_retornar_valor.py	receber tupla retornar valor
02_receber_lista_retornar_lista.py	receber lista retornar lista
03_mutabilidade_em_parametros.py	mutabilidade em parametros
04_evitar_parametro_mutavel_padrao.py	evitar parametro mutavel padrao
05_args_kwargs.py	args kwargs
06_buscar_cliente_no_banco.py	buscar cliente no banco
07_listar_produtos_retorna_lista.py	listar produtos retorna lista
08_funcao_produtos_estoque_baixo.py	funcao produtos estoque baixo

Trecho para leitura — exemplos/06_funcoes_parametro_retorno/04_evitar_parametro_mutavel_padrao.py

```
def registrar(nome, observacoes=None):
    if observacoes is None:
        observacoes = []
    observacoes.append(f"Cliente: {nome}")
    return observacoes

print(registrar("Ana"))
print(registrar("Bruno"))
```

Trecho para leitura — exemplos/06_funcoes_parametro_retorno/06_buscar_cliente_no_banco.py

```

from database import criar_conexao

def buscar_cliente(id_cliente):
    conexao = criar_conexao()
    cursor = conexao.cursor()
    cursor.execute("SELECT id_cliente, nome, cidade, uf FROM clientes WHERE id_cliente = %s", (id_cliente,))
    cliente = cursor.fetchone()
    cursor.close()
    conexao.close()
    return cliente

print(buscar_cliente(3))
print(buscar_cliente(999))

```

Como pensar nas atividades

- defina a responsabilidade da função em uma frase
- liste entradas e saída esperada
- decida se deve alterar a coleção recebida ou retornar outra
- para funções de banco, decida quem cria/fecha conexão e cursor e mantenha essa responsabilidade consistente

Arquivo	Foco	Tipo
01_tuple.py	Função recebe tuple	revisao aplicada
02_lista.py	Função recebe list	revisao aplicada
03_dict.py	Função recebe dict	revisao aplicada
04_mutabilidade.py	Efeito colateral em lista	conceito
05_copia.py	Evitar alteração do argumento	conceito
06_parametro_padrao.py	Evitar lista como valor padrão	conceito
07_args.py	*args	fixacao
08_kwargs.py	**kwargs	fixacao
09_buscar_cliente.py	Função de consulta	aplicacao com banco
10_listar_produtos.py	Função retorna lista	aplicacao com banco
11_estoque_baixo.py	Função com regra	aplicacao com banco
12_retorno_ou_print.py	return x print	escolha de recurso

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
receber e retornar	def f(dados): return resultado
padrão seguro	def f(itens=None): ...
posicionais extras	def f(*args):
nomeados extras	def f(**kwargs):
desempacotar tuple	f(*valores)
desempacotar dict	f(**dados)

7. Recursos de iteração e transformação: escrever menos sem esconder a lógica

Depois de dominar list, tuple, dict e set, Python oferece ferramentas para percorrer, combinar, transformar, agregar e ordenar coleções com mais clareza. O objetivo não é substituir todo for por uma expressão curta; é escolher uma forma que comunique melhor a intenção. Os exemplos desta pasta introduzem recursos que serão usados repetidamente nos capítulos seguintes.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

enumerate()

enumerate(iterável) produz pares (índice, valor). Ele é preferível a range(len(lista)) quando o objetivo é percorrer os elementos e também saber sua posição. O argumento start permite escolher o número inicial exibido sem alterar os índices reais da lista.

zip()

zip(a, b, ...) percorre iteráveis em paralelo e produz tuplas. É útil para associar colunas a valores, nomes a quantidades ou IDs a descrições. Por padrão, para o menor iterável; portanto tamanhos incompatíveis merecem atenção.

Comprehensions

List, dict e set comprehensions permitem construir coleções a partir de um percurso e, opcionalmente, um filtro. [p for p in produtos if p[2] < 30] é uma transformação declarativa. Quando a expressão precisa de múltiplas decisões, efeitos colaterais ou tratamento complexo, um for tradicional costuma ser mais legível.

any(), all() e agregações

any() responde se pelo menos um elemento é verdadeiro; all() exige que todos sejam verdadeiros. Ambos param assim que o resultado já é conhecido. sum(), min() e max() reduzem uma coleção a um valor; min/max aceitam key para escolher o critério sem alterar o elemento retornado.

sorted(), key, lambda e itemgetter

key transforma cada elemento em uma chave de comparação. lambda é adequada para expressões pequenas; operator.itemgetter() pode ser mais direto para posições/chaves conhecidas. sorted() é estável, o que permite ordenações em múltiplos critérios e preserva a ordem relativa quando as chaves são iguais.

Counter e defaultdict

collections.Counter conta ocorrências e fornece uma interface de dicionário especializada. defaultdict cria automaticamente um valor padrão por chave, como list para agrupamentos. Eles não substituem dict; são especializações úteis quando o padrão de uso é contar ou agrupar.

Quando usar

- precisar de índice e valor ao mesmo tempo
- combinar coleções paralelas
- criar novas coleções por transformação/filtro
- responder perguntas como existe algum?/todos?
- contar ou agrupar registros do banco
- ordenar por critérios complexos

Evite quando

- comprehensions longas que escondem regras
- lambda extensa com múltiplas condições
- Counter/defaultdict quando um dict simples já é suficiente
- zip sem conferir compatibilidade dos iteráveis

Erros e armadilhas comuns

- confundir índice de enumerate com valor do elemento
- esquecer que zip para no menor iterável
- usar comprehension apenas para executar efeitos colaterais
- usar all() em coleção vazia sem compreender que o resultado é True

No banco lojas_rede

Pedidos, canais e produtos dão sentido a esses recursos: Counter pode contar pedidos por canal; defaultdict pode agrupar pedidos por unidade; comprehensions podem extrair SKUs/preços; sorted() pode ordenar produtos

carregados. A escolha entre processar no banco e em Python continua válida: GROUP BY, WHERE e ORDER BY podem ser mais apropriados quando não há motivo para trazer todos os registros.

Como ler os exemplos da pasta

Arquivo	O que observar
01_enumerate.py	enumerate
02_zip.py	zip
03_list_comprehension.py	list comprehension
04_dict_set_comprehension.py	dict set comprehension
05_any_all.py	any all
06_sum_min_max.py	sum min max
07_sorted_key_lambda_itemgetter.py	sorted key lambda itemgetter
08_counter_defaultdict_com_banco.py	counter defaultdict com banco
09_comprehension_com_dados_do_banco.py	comprehension com dados do banco

Trecho para leitura — exemplos/07_recursos_iteracao_transformacao/01_enumerate.py

```
produtos = ["Shampoo Nutritivo 350 ml", "Condicionador Nutritivo 350 ml", "Perfume Floral 100 ml"]
for posicao, nome in enumerate(produtos, start=1):
    print(posicao, nome)
```

Trecho para leitura — exemplos/07_recursos_iteracao_transformacao/03_list_comprehension.py

```
precos = [39.90, 79.90, 249.90, 899.90]
com_desconto = [round(preco * 0.9, 2) for preco in precos]
baratos = [preco for preco in precos if preco < 100]
print(com_desconto)
print(baratos)
```

Trecho para leitura — exemplos/07_recursos_iteracao_transformacao/08_counter_defaultdict_com_banco.py

```
from collections import Counter, defaultdict
from database import criar_conexao

conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("""SELECT c.nome FROM pedidos p JOIN canais_venda c ON c.id_canal=p.id_canal""")
print("Pedidos por canal:", Counter(linha[0] for linha in cursor.fetchall()))

cursor.execute("SELECT id_fornecedor,nome FROM produtos ORDER BY id_fornecedor,nome")
por_fornecedor=defaultdict(list)
for id_fornecedor,nome in cursor.fetchall():
    por_fornecedor[id_fornecedor].append(nome)
print(dict(por_fornecedor))
cursor.close(); conexao.close()
```

Como pensar nas atividades

- descreva em português a operação: enumerar, combinar, filtrar, transformar, contar, agrupar ou ordenar
- escolha o recurso que expressa essa operação sem esconder o raciocínio
- se os dados estão no banco, avalie se o SQL poderia reduzir o volume antes de fetchall()

Arquivo	Foco	Tipo
01_enumerate.py	enumerate()	fixacao
02_zip.py	zip()	fixacao
03_list_comp.py	List comprehension	fixacao
04_filtro_comp.py	Comprehension com filtro	fixacao
05_dict_comp.py	Dict comprehension	fixacao
06_set_comp.py	Set comprehension	fixacao
07_any_all.py	any() e all()	fixacao
08_agregacoes.py	sum/min/max	fixacao
09_sorted_key.py	sorted() com key	fixacao
10_lambda_itemgetter.py	lambda x itemgetter	fixacao

Arquivo	Foco	Tipo
11_counter.py	COUNTER COM CANAIS	aplicação com banco
12_defaultdict.py	DEFAULTDICT COM FORNECEDORES	aplicação com banco

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
índice + valor	for i, x in enumerate(xs):
paralelo	for a, b in zip(as_, bs_):
list comp	[expr for x in xs if cond]
dict comp	{chave: valor for x in xs}
set comp	{expr for x in xs}
algum	any(cond for x in xs)
todos	all(cond for x in xs)
contagem	Counter(valores)
agrupamento	defaultdict(list)
ordenar	sorted(xs, key=...)

8. CRUD e persistência: quando a mudança precisa chegar ao MySQL

Até aqui, grande parte das operações alterou estruturas em memória. O CRUD marca a passagem para mudanças persistentes: SELECT lê; INSERT cria linhas; UPDATE altera linhas; DELETE remove linhas. A disciplina mantém a fronteira explícita porque ela evita uma confusão comum: método de coleção e comando SQL podem ter nomes/efeitos parecidos, mas operam em lugares diferentes.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

SELECT parametrizado

Parâmetros devem ser enviados separadamente da string SQL: `cursor.execute('SELECT ... WHERE id = %s', (id_cliente,))`. Isso evita concatenação manual, problemas de aspas e ataques de injeção. No MySQL Connector, o marcador é `%s` independentemente do tipo do valor.

INSERT, commit() e lastrowid

INSERT cria uma linha persistente quando a transação é confirmada. `cursor.lastrowid` permite obter o AUTO_INCREMENT gerado. Se o programa precisa usar esse ID para inserir itens relacionados na mesma transação, `lastrowid` é a ligação natural.

UPDATE, DELETE e rowcount

UPDATE e DELETE devem quase sempre ter um WHERE deliberado. `cursor.rowcount` indica quantas linhas foram afetadas e ajuda a detectar IDs inexistentes ou filtros incorretos. `rowcount` não substitui validação de regra de negócio, mas é um sinal útil.

executemany()

Quando a mesma instrução será executada com vários conjuntos de parâmetros, `executemany()` evita repetir chamadas manualmente e deixa a intenção clara. A confirmação continua obedecendo à transação: uma sequência de inserts só fica permanente após `commit()`, salvo se `autocommit` estiver ativo.

CRUD em funções

Encapsular `buscar_produto()`, `inserir_cliente()`, `atualizar_preco()` etc. ajuda a separar SQL da interface. A função deve receber parâmetros, usar placeholders e devolver dados úteis; o código de apresentação decide como exibi-los.

Quando usar

- a atividade exige que a alteração continue após novo SELECT/reexecução
- criar, alterar ou remover linhas reais
- precisar reaproveitar operações de persistência em funções

Evite quando

- usar f-string/concatenação para colocar dados do usuário no SQL
- executar UPDATE/DELETE sem conferir o WHERE
- dar commit após cada pequena etapa de uma operação que deveria ser atômica

Erros e armadilhas comuns

- confundir `append` com `INSERT`
- confundir alteração de dict com `UPDATE`
- esquecer commit e acreditar que a alteração foi confirmada
- montar tuple de um parâmetro sem a vírgula
- não conferir `rowcount`

No banco lojas_rede

Os exemplos trabalham diretamente com clientes, produtos e pedidos. Para preservar o banco didático, muitos exemplos usam dados temporários ou removem o que criaram; ainda assim, o princípio é real. Sempre que a atividade pede demonstração sem mudar o estado inicial, prefira `rollback()` ao final ou execute `reiniciar_banco.py` depois.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de `SELECT` é local ao processo Python. Operações como `append()`, `pop()`, `add()`, alteração de dict, `popleft()`, `heappop()` ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_select_parametrizado.py	select parametrizado
02_insert_commit_lastrowid.py	insert commit lastrowid
03_update_commit_rowcount.py	update commit rowcount
04_delete_commit.py	delete commit
05_executemany.py	executemany
06_alterar_dict_vs_update.py	alterar dict vs update
07_injection_parametros.py	injection parametros
08_crud_em_funcoes.py	crud em funcoes

Trecho para leitura — exemplos/08_crud_persistencia/01_select_parametrizado.py

```
from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

id_cliente = 3
cursor.execute("SELECT id_cliente, nome, cidade FROM clientes WHERE id_cliente = %s", (id_cliente,))
print(cursor.fetchone())

cursor.close()
conexao.close()
```

Trecho para leitura — exemplos/08_crud_persistencia/02_insert_commit_lastrowid.py

```

from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()
cpf_teste = "900.000.000-01"
cursor.execute("DELETE FROM clientes WHERE cpf_cnpj = %s", (cpf_teste,))
conexao.commit()

cursor.execute("""
    INSERT INTO clientes (nome, cpf_cnpj, email, origem, cidade, uf)
    VALUES (%s, %s, %s, %s, %s, %s)
""", ("Cliente Teste", cpf_teste, "teste@email.com", "AULA", "Sete Lagoas", "MG"))
conexao.commit()
print("ID gerado:", cursor.lastrowid)

cursor.execute("SELECT id_cliente, nome FROM clientes WHERE cpf_cnpj = %s", (cpf_teste,))
print(cursor.fetchone())

cursor.execute("DELETE FROM clientes WHERE cpf_cnpj = %s", (cpf_teste,))
conexao.commit()
cursor.close()
conexao.close()

```

Trecho para leitura — exemplos/08_crud_persistencia/03_update_commit_rowcount.py

```

from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()
cursor.execute("SELECT nome FROM clientes WHERE id_cliente = 1")
nome_original = cursor.fetchone()[0]

cursor.execute("UPDATE clientes SET nome = %s WHERE id_cliente = %s", ("Ana Teste", 1))
print("Linhas afetadas:", cursor.rowcount)
conexao.commit()

cursor.execute("SELECT nome FROM clientes WHERE id_cliente = 1")
print("Persistido:", cursor.fetchone()[0])

cursor.execute("UPDATE clientes SET nome = %s WHERE id_cliente = %s", (nome_original, 1))
conexao.commit()
cursor.close()
conexao.close()

```

Como pensar nas atividades

- identifique se a operação é leitura ou persistência
- escolha o comando SQL correspondente
- use parâmetros separados da string
- decida qual é a fronteira da transação antes de chamar commit()

Arquivo	Foco	Tipo
01_select.py	SELECT parametrizado	fixacao com banco
02_insert.py	INSERT + commit	persistencia
03_lastrowid.py	lastrowid	persistencia
04_update.py	UPDATE + rowcount	persistencia
05_delete.py	DELETE + rowcount	persistencia
06_executemany.py	executemany()	persistencia
07_parametros.py	Parâmetros SQL	conceito
08_dict_update.py	dict x UPDATE	memoria x persistencia
09_list_delete.py	pop x DELETE	memoria x persistencia
10_where_ou_filtro.py	WHERE x filtro em Python	escolha de recurso
11_commit_esquecido.py	Esquecer commit	diagnostico

Arquivo	Foco	Tipo
12_crud_funcoes.py	CRUD em funções	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
consulta	<code>cursor.execute(sql, params); cursor.fetchone()/fetchall()</code>
inserir	<code>INSERT ... VALUES (%s, ...)</code>
id gerado	<code>cursor.lastrowid</code>
alterar	<code>UPDATE ... SET ... WHERE ...</code>
excluir	<code>DELETE ... WHERE ...</code>
afetadas	<code>cursor.rowcount</code>
vários	<code>cursor.executemany(sql, dados)</code>
confirmar	<code>conexao.commit()</code>

9. Transações: atomicidade, commit, rollback e bloqueio de estoque

Uma venda não é apenas “tirar quantidade do estoque”. Ela pode criar um pedido, inserir itens e atualizar estoque. Se uma etapa falhar e as anteriores ficarem gravadas, o banco entra em estado inconsistente. A transação trata esse conjunto como uma unidade: ou todas as alterações são confirmadas ou nenhuma deve permanecer.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

commit() e rollback()

`commit()` confirma as alterações da transação. `rollback()` abandona as alterações ainda não confirmadas. Isso não “volta no tempo” para objetos Python: se uma list foi alterada em memória, `rollback()` não desfaz `append/pop` nem restaura valores locais. Banco e memória continuam sendo estados separados.

try/except como fronteira transacional

Um padrão comum é: iniciar as operações, validar, executar os comandos; se tudo der certo, `commit()`; se qualquer etapa falhar, `rollback()` e tratar/propagar a exceção. O `commit` deve ficar no final da unidade lógica, não no meio.

SELECT ... FOR UPDATE

Quando a transação lê uma linha que será usada para decidir uma atualização, `FOR UPDATE` pode bloquear a linha até `commit/rollback`, evitando que outra transação altere aquele estoque no intervalo. É um mecanismo de concorrência do banco; não transforma a tuple retornada em objeto “vivo”.

Autocommit

Com `autocommit` habilitado, cada instrução pode ser confirmada automaticamente e a possibilidade de agrupar várias instruções diminui. Para atividades de transação, mantenha o comportamento controlado e confirme explicitamente. Sempre saiba qual modo a conexão está utilizando.

Quando usar

- uma operação envolve várias tabelas/linhas que devem permanecer consistentes
- a leitura orienta uma atualização concorrente de estoque
- é necessário garantir tudo-ou-nada

Evite quando

- `commit` cedo demais, antes de concluir a operação inteira
- capturar erro e continuar sem `rollback`

- misturar alterações de banco e memória imaginando que rollback desfaz ambas

Erros e armadilhas comuns

- esquecer que rollback só afeta a transação do banco
- usar FOR UPDATE fora de uma transação útil
- deixar conexão aberta mantendo bloqueios
- confundir atomicidade com validação de regras

No banco lojas_rede

A simulação natural em lojas_rede é uma venda: selecionar estoque com FOR UPDATE, criar pedido, inserir item e reduzir estoque. Nas atividades didáticas, rollback() ao final permite praticar sem destruir o estado inicial. Depois do rollback, um novo SELECT deve confirmar que o banco voltou ao estado anterior, mesmo que variáveis Python ainda contenham valores calculados durante a tentativa.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como append(), pop(), add(), alteração de dict, popleft(), heappop() ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_rollback_desfaz_update.py	rollback desfaz update
02_commit_confirma.py	commit confirma
03_try_except_rollback.py	try except rollback
04_for_update_estoque.py	for update estoque
05_transacao_multiplas_tabelas_simulada.py	transacao multiplas tabelas simulada
06_estado_em_memoria_apos_rollback.py	estado em memoria apos rollback

Trecho para leitura — exemplos/09_transacoes/03_try_except_rollback.py

```
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
try:
    cursor.execute("UPDATE estoques SET quantidade=quantidade-1 WHERE id_unidade=1 AND id_produto=4")
    # FK inválida força falha para demonstrar rollback.
    cursor.execute("INSERT INTO itens_pedido (id_pedido,id_produto,quantidade,preco_unitario) VALUES
(99999,4,1,129.90))
    conexao.commit()
except Exception as erro:
    conexao.rollback()
    print(type(erro).__name__, "-> rollback executado")
finally:
    cursor.close(); conexao.close()
```

Trecho para leitura — exemplos/09_transacoes/04_for_update_estoque.py

```
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
try:
    cursor.execute("SELECT quantidade FROM estoques WHERE id_unidade=%s AND id_produto=%s FOR
UPDATE",(1,4))
    print("Estoque bloqueado para decisão:",cursor.fetchone()[0])
    conexao.rollback()
finally:
    cursor.close(); conexao.close()
```

Trecho para leitura — exemplos/09_transacoes/06_estado_em_memoria_apos_rollback.py

```

from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("""SELECT p.id_produto,p.nome,e.quantidade FROM produtos p JOIN estoques e ON
e.id_produto=p.id_produto WHERE p.id_produto=1 AND e.id_unidade=1""")
registro=cursor.fetchone(); memoria=list(registro)
memoria[2]-=5
cursor.execute("UPDATE estoques SET quantidade=quantidade-5 WHERE id_unidade=1 AND id_produto=1")
conexao.rollback()
print("Objeto em memória:",memoria)
cursor.execute("""SELECT p.id_produto,p.nome,e.quantidade FROM produtos p JOIN estoques e ON
e.id_produto=p.id_produto WHERE p.id_produto=1 AND e.id_unidade=1""")
print("Banco após rollback:",cursor.fetchone())
cursor.close(); conexao.close()

```

Como pensar nas atividades

- liste todas as alterações que formam uma única operação de negócio
- coloque commit somente depois da última etapa bem-sucedida
- defina o que fazer se qualquer etapa falhar
- se houver estoque concorrente, considere FOR UPDATE
- após rollback, consulte novamente o banco em vez de confiar apenas nas variáveis locais

Arquivo	Foco	Tipo
01_rollback.py	Rollback	fixacao
02_commit.py	Commit	fixacao
03_memoria_rollback.py	Rollback não desfaz objeto Python	conceito
04_erro_meio.py	Erro no meio da transação	aplicacao com banco
05_for_update.py	SELECT FOR UPDATE	aplicacao com banco
06_venda_simulada.py	Venda simulada atômica	integração
07_commit_parcial.py	Por que não commitar cedo demais?	diagnostico
08_try_transacao.py	try/except em transação	escolha de recurso
09_pedido_itens.py	Pedido e itens	integração
10_autocommit.py	Autocommit	diagnostico
11_frenteira.py	Frenteira da transação	conceito
12_repetir_consulta.py	Verificar após rollback	aplicacao

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
confirmar	conexao.commit()
desfazer	conexao.rollback()
bloquear leitura	SELECT ... FOR UPDATE
padrão	try: comandos; commit() / except: rollback(); raise ou tratar

10. Complexidade: comparar crescimento, não apenas cronometrar

Estruturas diferentes resolvem a mesma tarefa com custos diferentes. Complexidade assintótica descreve como o número de operações cresce quando o tamanho da entrada aumenta. Ela não substitui medições reais nem considera todos os detalhes de hardware, mas oferece uma linguagem para justificar escolhas como list versus set/dict, pop(0) versus deque.popleft() e busca linear versus binária.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

O(1), O(n) e O(n²)

Acesso por índice em list é tratado como O(1): aumentar a lista não exige percorrer os elementos para chegar a uma posição conhecida. Uma busca linear pode examinar n elementos, portanto O(n). Dois laços que comparam todos com todos normalmente produzem crescimento quadrático O(n²). Big-O ignora constantes e foca a tendência de crescimento.

Tempo e memória

Criar um dict por SKU consome memória adicional e custa O(n) para construir, mas torna buscas repetidas médias O(1). Esse é um trade-off: gastar memória e preparação para ganhar velocidade de acesso. A melhor escolha depende de quantas buscas ocorrerão e do tamanho/tempo de vida do conjunto.

Medição com timeit

timeit permite comparar pequenas operações, mas resultados isolados não provam uma classe de complexidade. Ruído, cache, implementação em C e tamanho pequeno podem enganar. Use medição para observar comportamento e Big-O para raciocinar sobre crescimento.

SQL e volume de dados

Trazer milhares de linhas e filtrar em Python também tem custo de rede, memória e CPU. Um WHERE adequado pode reduzir o conjunto antes da materialização. A análise de complexidade da aplicação precisa considerar onde o trabalho acontece, não apenas o laço Python.

Quando usar

- comparar alternativas para coleções grandes ou operações repetidas
- justificar índices em memória, filas, heaps e buscas
- entender por que determinadas operações nativas são recomendadas

Evite quando

- tratar Big-O como tempo em segundos
- comparar algoritmos sem considerar pré-condições e custo de preparação
- otimizar prematuramente um conjunto minúsculo ignorando clareza

Erros e armadilhas comuns

- dizer que dict é sempre O(1) sem mencionar caso médio e colisões
- ignorar o custo de ordenar antes da busca binária
- ignorar memória extra de índices
- confundir tempo do banco com tempo do Python

No banco lojas_rede

Produtos do banco fornecem um conjunto conhecido para contar comparações e construir índices temporários. As atividades também comparam filtrar com WHERE e filtrar depois do fetchall(). A pergunta central é sempre: qual operação cresce com n e qual preparação está sendo paga para acelerar as consultas seguintes?

Como ler os exemplos da pasta

Arquivo	O que observar
01_o1_indice_lista.py	o1 indice lista
02_on_busca_linear_contagem.py	on busca linear contagem
03_on2_dois_lacos.py	on2 dois lacos
04_dict_lookup_vs_lista.py	dict lookup vs lista
05_medir_sem_tirar_conclusao_absoluta.py	medir sem tirar conclusao absoluta
06_complexidade_com_produtos_do_banco.py	complexidade com produtos do banco

Trecho para leitura — exemplos/10_complexidade/04_dict_lookup_vs_lista.py

```
clientes = [(1, "Ana"), (2, "Bruno"), (3, "Carla")]
por_id = {id_cliente: nome for id_cliente, nome in clientes}
print(por_id[3])
```

Trecho para leitura — exemplos/10_complexidade/05_medir_sem_tirar_conclusao_absoluta.py

```
from timeit import timeit

lista = list(range(10_000))
conjunto = set(lista)
print("in em lista:", timeit(lambda: 9999 in lista, number=1000))
print("in em set:", timeit(lambda: 9999 in conjunto, number=1000))
```

Como pensar nas atividades

- defina n: número de produtos, rotas, solicitações?
- conte a estrutura do algoritmo: acesso direto, um percurso, percursos aninhados
- inclua custos de preparação como sorted() ou construção de dict
- compare tempo e memória, não apenas uma dimensão

Arquivo	Foco	Tipo
01_o1.py	$O(1)$	conceito
02_on.py	$O(n)$	conceito
03_on2.py	$O(n^2)$	conceito
04_list_set.py	Pertencimento em list x set	comparacao
05_pop0_deque.py	pop(0) x popleft()	comparacao
06_busca_produtos.py	Contar comparações em produtos	aplicacao com banco
07_dict_index.py	Construir índice em memória	escolha de recurso
08_tradeoff.py	Tempo x memória	conceito
09_big_o_nao_tempo.py	Big-O não é cronômetro	diagnostico
10_crescimento.py	Tabela de crescimento	aplicacao
11_sql_where.py	Filtrar no banco ou em memória	escolha de recurso
12_complexidade_pipeline.py	Pipeline	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
acesso list[i]	$O(1)$
x in list	$O(n)$
dict/set lookup	$O(1)$ médio
pop(0) em list	$O(n)$
deque.popleft()	$O(1)$
busca binária	$O(\log n)$
heap push/pop	$O(\log n)$
sort típico	$O(n \log n)$

11. Pilhas: LIFO, histórico e desfazer

Pilha é uma política de acesso: o último elemento inserido é o primeiro a sair (LIFO). Python não exige uma classe especial para a forma básica; list.append() funciona como push e list.pop() como pop no final, ambos eficientes. O banco oferece um exemplo natural: eventos de status de um pedido podem ser carregados cronologicamente e processados do mais recente para o mais antigo.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Operações fundamentais

push acrescenta ao topo; pop remove o topo; peek apenas consulta o topo; is_empty verifica se há elementos; size informa a quantidade. Com list, topo = pilha[-1] e len(pilha) fornece o tamanho. Antes de pop, trate pilha vazia para evitar IndexError.

Histórico e desfazer

Uma pilha é apropriada quando a operação seguinte deve agir sobre o evento mais recente: desfazer alterações, voltar na navegação, processar chamadas aninhadas ou validar delimitadores. Nem todo histórico é uma pilha persistida; frequentemente o banco armazena eventos em ordem e o programa escolhe tratá-los com LIFO em memória.

Parênteses balanceados

O algoritmo clássico empilha delimitadores de abertura e, ao encontrar um fechamento, compara com o topo. Ele mostra que pilha não é apenas uma lista usada de qualquer maneira; a restrição de acesso ao topo é parte do modelo.

Quando usar

- último a entrar deve ser o primeiro a sair
- desfazer operações
- processar histórico recente
- rastrear estruturas aninhadas

Evite quando

- quando o primeiro a entrar precisa sair primeiro; use fila
- quando precisa retirar por prioridade; use heap
- quando acesso arbitrário a qualquer posição é parte essencial da tarefa

Erros e armadilhas comuns

- chamar pop() em pilha vazia
- usar insert(0)/pop(0) desnecessariamente
- acreditar que pop() do histórico em memória executa DELETE
- chamar uma list de pilha mas acessar/remover elementos arbitrariamente

No banco lojas_rede

historico_status_pedido é carregado e ordenado por data. Ao copiar os registros para uma list e usar pop(), os eventos deixam apenas a pilha em memória. A tabela continua intacta. Esse é o mesmo princípio usado em todas as estruturas: o banco fornece fatos persistidos; a estrutura organiza o processamento temporário.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como append(), pop(), add(), alteração de dict, popleft(), heappop() ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_lifo_com_list.py	lifo com list
02_is_empty_size_peek.py	is empty size peek
03_historico_pedido_do_banco.py	historico pedido do banco
04_pop_memoria_nao_exclui_historico.py	pop memoria nao exclui historico
05_desfazer_conceitual.py	desfazer conceitual
06_validar_parenteses_com_pilha.py	validar parenteses com pilha

Trecho para leitura — exemplos/11_pilhas/01_lifo_com_list.py

```

pilha = []
pilha.append("PENDENTE")
pilha.append("PAGO")
pilha.append("SEPARACAO")
print("Topo:", pilha[-1])
print("Sai:", pilha.pop())
print(pilha)

```

Trecho para leitura — exemplos/11_pilhas/03_historico_pedido_do_banco.py

```

from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

cursor.execute("""
    SELECT status, observacao, data_evento
    FROM historico_status_pedido
    WHERE id_pedido = 1
    ORDER BY data_evento
""")
pilha = cursor.fetchall()
print("Topo:", pilha[-1])
while pilha:
    print("pop ->", pilha.pop())

cursor.close()
conexao.close()

```

Como pensar nas atividades

- pergunte qual elemento deve ser o próximo: o mais recente?
- defina claramente qual extremo é o topo
- trate a estrutura vazia
- se o dado veio do banco, separe a operação LIFO da persistência

Arquivo	Foco	Tipo
01_push_pop.py	push/pop com list	fixacao
02_peek.py	Peek	fixacao
03_empty.py	Vazia e tamanho	fixacao
04_historico.py	Histórico como pilha	aplicacao com banco
05_pop_nao_delete.py	pop() não é DELETE	memoria x persistencia
06_desfazer.py	Pilha de desfazer	aplicacao
07_parenteses.py	Parênteses balanceados	aplicacao
08_pilha_ou_fila.py	Pilha ou fila?	escolha de recurso
09_topo.py	Acesso restrito	conceito
10_movimentos.py	Eventos recentes de um pedido	aplicacao com banco
11_pop_vazia.py	Pop em pilha vazia	diagnostico
12_funcao_stack.py	Encapsular operações	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
push	<code>pilha.append(x)</code>
pop	<code>x = pilha.pop()</code>
peek	<code>pilha[-1]</code>

Recurso	Forma
vazia	not pilha
tamanho	len(pilha)

12. Filas e deque: FIFO, duas extremidades e reposição por chegada

Fila implementa FIFO: o primeiro que entra é o primeiro que sai. Uma list pode demonstrar o conceito com `append()` e `pop(0)`, mas `pop(0)` desloca os elementos restantes e custa $O(n)$. `collections.deque` é a estrutura prática para inserções e remoções eficientes nas duas extremidades.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

deque básico

`deque.append(x)` insere à direita e `popleft()` remove à esquerda, formando FIFO. `appendleft()` e `pop()` permitem trabalhar no sentido oposto. `rotate(k)` desloca elementos circularmente e pode representar escalonamento round-robin.

Fila a partir do banco

`solicitacoes_reposicao` possui `data_solicitacao`. O SELECT deve usar ORDER BY `data_solicitacao` quando a regra é ordem de chegada. Depois de `fetchall()`, `deque(registros)` cria a fila em memória. A ordem da fila depende da consulta; deque não sabe qual campo representa a data.

FIFO versus prioridade

FIFO e prioridade são políticas diferentes. Uma solicitação mais recente com prioridade 1 pode ser atendida antes de uma antiga com prioridade 4 se a regra de negócio for urgência. O capítulo de heap apresenta a estrutura adequada para essa outra política.

Quando usar

- ordem de chegada determina o próximo item
- precisa inserir/remover nas extremidades
- round-robin ou janela deslizante simples
- BFS, que exige retirar do início e inserir no fim

Evite quando

- usar `list.pop(0)` em filas grandes
- usar FIFO quando prioridade deve dominar
- usar deque como substituto automático de list para qualquer coleção

Erros e armadilhas comuns

- esquecer ORDER BY antes de montar fila com dados do banco
- confundir `popleft()` com UPDATE de status
- assumir que deque está ordenado por algum campo
- usar `rotate()` sem compreender que apenas reorganiza memória

No banco lojas_rede

Os exemplos 04 a 07 carregam solicitações reais. `popleft()` escolhe e remove o próximo elemento da deque, mas o status da linha permanece PENDENTE. Para registrar que uma reposição começou, é necessário UPDATE `solicitacoes_reposicao SET status = ... WHERE id_solicitacao = ...` e `commit()`.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como `append()`, `pop()`, `add()`, alteração de dict, `popleft()`, `heappop()` ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
---------	----------------

Arquivo	O que observar
01_fifo_com_lista.py	fifo com lista
02_deque_append_popleft.py	deque append popleft
03_appendleft_pop.py	appendleft pop
04_solicitacoes_reposicao_fifo_do_banco.py	solicitacoes reposicao fifo do banco
05_popleft_nao_atualiza_status.py	popleft nao atualiza status
06_rotacao_deque.py	rotacao deque
07_comparar_fifo_e_prioridade.py	comparar fifo e prioridade

Trecho para leitura — exemplos/12_filas_deque/02_deque_append_popleft.py

```
from collections import deque

fila = deque(["Ana", "Bruno", "Carla"])
fila.append("Diego")
print("Atendido:", fila.popleft())
print(fila)
```

Trecho para leitura — exemplos/12_filas_deque/04_solicitacoes_reposicao_fifo_do_banco.py

```
from collections import deque
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("""
    SELECT s.id_solicitacao,u.nome,p.sku,s.prioridade,s.data_solicitacao
    FROM solicitacoes_reposicao s
    JOIN unidades u ON u.id_unidade=s.id_unidade
    JOIN produtos p ON p.id_produto=s.id_produto
    WHERE s.status='PENDENTE' ORDER BY s.data_solicitacao
""")
fila=deque(cursor.fetchall())
print("Próxima por chegada:",fila[0])
print("Retirada apenas da memória:",fila.popleft())
cursor.close(); conexao.close()
```

Como pensar nas atividades

- identifique a política: chegada ou prioridade?
- garanta a ordem da consulta antes de criar a fila
- use deque quando as extremidades são operadas repetidamente
- se o próximo item muda de status real, planeje a operação SQL separadamente

Arquivo	Foco	Tipo
01_fifo_lista.py	FIFO com list	fixacao
02_deque.py	FIFO com deque	fixacao
03_extremos.py	Duas extremidades	fixacao
04_fila_reposicoes.py	FILA DE REPOSIÇÕES	aplicação com banco
05_popleft_nao_update.py	popleft() não muda status	memoria x persistencia
06_update_status.py	Persistir início de atendimento	persistencia
07_fifo_prioridade.py	FIFO X PRIORIDADE	comparação
08_deque_vs_list.py	deque ou list?	escolha de recurso
09_round_robin.py	rotate()	aplicacao
10_fila_vazia.py	Fila vazia	diagnostico
11_funcao_fila.py	Funções de fila	integração
12_ordem_banco.py	ORDER BY e fila	conceito

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
criar	<code>from collections import deque; fila = deque()</code>
entrar	<code>fila.append(x)</code>
sair FIFO	<code>fila.popleft()</code>
esquerda	<code>fila.appendleft(x)</code>
direita	<code>fila.pop()</code>
rotacionar	<code>fila.rotate(k)</code>

13. Busca linear: procurar sem exigir ordenação

Busca linear percorre elementos um a um até encontrar o alvo ou terminar a coleção. É simples, funciona em dados não ordenados e é adequada para poucas buscas em conjuntos pequenos. O custo no pior caso é $O(n)$. Os exemplos evoluem de valores literais para produtos carregados do banco e comparam busca em memória com `SELECT ... WHERE`.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Primeiro, posição ou todos

Uma busca pode ter objetivos diferentes: retornar o primeiro registro, a posição em que apareceu ou todos os registros que atendem ao critério. Esses contratos mudam o algoritmo. Para o primeiro, `return/break` encerra cedo; para todos, o percurso precisa continuar e acumular resultados.

Sentinela de ausência

`None` é uma escolha comum para “não encontrado” quando `None` não é um resultado válido. Em busca de posição, `-1` também é comum, mas o contrato deve ser documentado. Não confunda ausência em uma coleção já carregada com `fetchone() == None`; são situações semelhantes semanticamente, mas produzidas em camadas diferentes.

Busca por trecho

Crítérios não precisam ser igualdade exata. É possível usar `in` em strings, normalização com `lower()` ou várias condições. Quanto mais complexa a regra, mais importante fica separar a função de busca dos dados.

Python versus WHERE

Se o programa já possui os produtos em memória e fará várias análises, uma busca linear pode ser natural. Se precisa de um único produto por SKU e ainda não carregou o catálogo, `WHERE sku = %s` é mais direto e evita transferência desnecessária.

Quando usar

- coleção não está ordenada
- há poucas buscas
- critério é flexível ou complexo e os dados já estão em memória
- quer ensinar/medir comparações

Evite quando

- muitas buscas repetidas no mesmo conjunto; um dict pode ser melhor
- coleção ordenada grande em que busca binária é aplicável
- consultar todo o banco só para localizar uma linha por chave

Erros e armadilhas comuns

- esquecer `break/return` quando só o primeiro importa
- usar variável de resultado sem inicializar caso não encontre
- alterar a coleção enquanto percorre
- confundir posição da lista com ID do banco

No banco lojas_rede

O catálogo de produtos pode ser carregado por fetchall() e pesquisado por SKU. Depois o exemplo faz a mesma pergunta ao banco com WHERE. A comparação não tem um vencedor universal; ela ensina a decidir onde os dados já estão e quantas operações serão feitas.

Como ler os exemplos da pasta

Arquivo	O que observar
01_buscar_primeiro.py	buscar primeiro
02_buscar_posicao.py	buscar posicao
03_buscar_todos.py	buscar todos
04_funcao_busca_linear.py	funcao busca linear
05_busca_linear_produtos_banco.py	busca linear produtos banco
06_comparar_busca_python_e_where.py	comparar busca python e where

Trecho para leitura — exemplos/13_busca_linear/04_funcao_busca_linear.py

```
def buscar_por_sku(produtos, sku):
    for produto in produtos:
        if produto[0] == sku:
            return produto
    return None

print(buscar_por_sku([("CAB-SHA-001", "Shampoo"), ("PERF-FEM-001", "Perfume")], "PERF-FEM-001"))
```

Trecho para leitura — exemplos/13_busca_linear/06_comparar_busca_python_e_where.py

```
from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

cursor.execute("SELECT sku, nome FROM produtos")
produtos = cursor.fetchall()
resultado_python = next((p for p in produtos if p[0] == "PEL-FAC-001"), None)
cursor.execute("SELECT sku, nome FROM produtos WHERE sku=%s", ("PEL-FAC-001",))
resultado_sql = cursor.fetchone()
print("Python:", resultado_python)
print("SQL:", resultado_sql)

cursor.close()
conexao.close()
```

Como pensar nas atividades

- defina exatamente o que deve ser retornado
- determine o critério e o valor de ausência
- decida se pode encerrar cedo
- pergunte se os dados já estão em memória ou se o banco pode responder diretamente

Arquivo	Foco	Tipo
01_primeiro.py	Encontrar primeiro	fixacao
02_posicao.py	Retornar posição	fixacao
03_todos.py	Encontrar todos	fixacao
04_sentinela.py	Não encontrado	fixacao
05_sku.py	Buscar SKU no banco carregado	aplicacao com banco
06_substring.py	Busca por trecho	aplicacao com banco
07_where.py	Busca linear x WHERE	comparacao
08_repetidas.py	Buscas repetidas	escolha de recurso
09_break.py	Uso de break	diagnostico
10_funcao.py	Função reutilizável	integração
11_ordem.py	Busca linear não exige ordenação	conceito

Arquivo	Foco	Tipo
12_resultado.py	Resultado é referência/valor em memória	memoria x persistencia

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
padrão	for item in itens: if criterio(item): return item
posição	for i, item in enumerate(itens): ...
todos	resultados.append(item)
ausência	return None

14. Busca binária e bisect: reduzir pela metade exige ordem

Busca binária descarta metade do intervalo a cada comparação e alcança $O(\log n)$, mas só funciona corretamente quando os dados estão ordenados pelo mesmo critério usado na busca. Essa pré-condição é tão importante quanto o algoritmo. O módulo bisect oferece ferramentas prontas para localizar posições de inserção em listas ordenadas.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Algoritmo iterativo

Mantêm-se limites início e fim. Calcula-se o meio; se o valor é igual, termina; se o alvo é menor, move o fim; se maior, move o início. O laço continua enquanto início $\leq$ fim. Erros de $+1/-1$ podem causar repetição infinita ou perder elementos, por isso contar casos pequenos manualmente é uma boa estratégia.

Custo de ordenar

Se a lista ainda não está ordenada, é preciso pagar o custo da ordenação antes. Para uma única busca, ordenar $O(n \log n)$ apenas para buscar $O(\log n)$ pode ser pior que uma busca linear $O(n)$. Quando a coleção será consultada muitas vezes e permanecerá estável, a preparação pode compensar.

bisect_left() e insert()

bisect_left(lista, x) devolve a posição em que x poderia ser inserido sem quebrar a ordem, antes dos valores iguais. É necessário verificar se o índice está dentro da lista e se lista[i] == x para confirmar existência. insert() insere mantendo a ordem, mas inserir em list ainda pode deslocar elementos e custar $O(n)$.

Duplicatas e chave

Buscar por preço pode encontrar vários produtos com o mesmo valor. Uma busca binária simples encontra uma ocorrência, não necessariamente todas. Para dados compostos, é comum ordenar por uma chave extra ou manter uma sequência separada de chaves.

Quando usar

- dados já estão ordenados
- haverá muitas buscas na mesma coleção estável
- precisa localizar posição de inserção em sequência ordenada

Evite quando

- coleção muda constantemente e cada inserção exige deslocamentos
- apenas uma busca em dados não ordenados
- critério de busca não corresponde à ordenação

Erros e armadilhas comuns

- esquecer a pré-condição de ordenação

- confundir índice do banco com índice da lista
- usar `bisect_left` e assumir que encontrou sem validar igualdade
- ignorar duplicatas

No banco lojas_rede

O exemplo com SKUs ordena os valores carregados e usa `bisect_left`. Isso é um índice em memória e não tem relação direta com os índices internos do MySQL. Um índice SQL é uma estrutura mantida pelo SGBD; `bisect` opera somente sobre a list Python que existe naquele processo.

Como ler os exemplos da pasta

Arquivo	O que observar
01_algoritmo_iterativo.py	algoritmo iterativo
02_pre_condicao_ordenacao.py	pre condicao ordenacao
03_contar_comparacoes.py	contar comparacoes
04_bisect_left.py	bisect left
05_insort_mantem_ordem.py	insort mantém ordem
06_busca_binaria_skus_banco.py	busca binaria skus banco

Trecho para leitura — exemplos/14_busca_binaria/01_algoritmo_iterativo.py

```
def busca_binaria(valores, alvo):
    inicio = 0
    fim = len(valores) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2
        if valores[meio] == alvo:
            return meio
        if valores[meio] < alvo:
            inicio = meio + 1
        else:
            fim = meio - 1
    return None

print(busca_binaria([10, 20, 30, 40, 50], 40))
```

Trecho para leitura — exemplos/14_busca_binaria/04_bisect_left.py

```
from bisect import bisect_left

valores = [10, 20, 30, 40, 50]
pos = bisect_left(valores, 30)
print(pos, valores[pos])
```

Trecho para leitura — exemplos/14_busca_binaria/06_busca_binaria_skus_banco.py

```
from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

from bisect import bisect_left

cursor.execute("SELECT sku, nome FROM produtos ORDER BY sku")
produtos = cursor.fetchall()
skus = [p[0] for p in produtos]
alvo = "PEL-FAC-001"
pos = bisect_left(skus, alvo)
if pos < len(skus) and skus[pos] == alvo:
    print(produtos[pos])
else:
    print("Não encontrado")

cursor.close()
conexao.close()
```

Como pensar nas atividades

- confirme que a coleção está ordenada pela chave correta
- calcule se o custo de ordenar é justificável
- defina o comportamento para duplicatas
- se os dados permanecem no banco, avalie se uma consulta indexada é mais apropriada

Arquivo	Foco	Tipo
01_algoritmo.py	Implementar busca binária	fixacao
02_ordenacao.py	Pré-condição	conceito
03_comparacoes.py	Contar comparações	fixacao
04_bisect_left.py	bisect_left	fixacao
05_insort.py	insort	fixacao
06_sku.py	SKUs ordenados	aplicacao com banco
07_preco.py	Preço com duplicatas	aplicacao com banco
08_linear.py	Linear x binária	comparacao
09_custo_ordenar.py	Vale ordenar?	escolha de recurso
10_indices.py	Limites inicio/fim	diagnostico
11_sql_index.py	Índice do banco não é bisect	conceito
12_funcao_sku.py	Função de busca binária	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
meio	<code>meio = (inicio + fim) // 2</code>
biblioteca	<code>from bisect import bisect_left, insort</code>
posição	<code>i = bisect_left(valores, alvo)</code>
confirmar	<code>i < len(valores) and valores[i] == alvo</code>
inserir ordenado	<code>insort(valores, x)</code>

15. Ordenação: algoritmos didáticos, sort/sorted e ORDER BY

Ordenar é reorganizar elementos segundo um critério. A disciplina implementa Bubble, Selection e Insertion Sort para tornar visíveis comparações e trocas; em aplicações reais, normalmente se usa `sort()`, `sorted()` ou `ORDER BY`. O objetivo é compreender o custo e as propriedades, não substituir implementações eficientes da biblioteca.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Bubble Sort

Compara vizinhos e troca quando estão fora de ordem. A cada passagem, um dos maiores elementos migra para o final. A versão com flag para quando nenhuma troca ocorre, melhorando casos já ordenados, embora o pior caso permaneça $O(n^2)$.

Selection Sort

Em cada posição, procura o menor elemento restante e o coloca ali. Faz número previsível de comparações e poucas trocas, mas continua $O(n^2)$. Em sua forma comum não é estável.

Insertion Sort

Mantém uma parte inicial ordenada e insere cada novo elemento na posição correta, deslocando maiores. É simples e pode ser eficiente em listas pequenas ou quase ordenadas, embora o pior caso seja $O(n^2)$.

sort(), sorted(), key e estabilidade

sort() altera list; sorted() produz nova list a partir de qualquer iterável. key define o valor usado na comparação e reverse inverte a ordem. A ordenação de Python é estável: registros com a mesma chave preservam a ordem relativa, propriedade útil em ordenações por múltiplos critérios.

ORDER BY versus sorted()

ORDER BY faz o banco devolver os dados já ordenados. sorted() é útil quando os registros já estão em memória ou quando o critério depende de transformação local. Evite buscar tudo sem ordem e reordenar em Python se o banco poderia devolver apenas o recorte necessário na ordem correta.

Quando usar

- precisa exibir/processar dados em uma ordem
- quer estudar comparações/trocas e complexidade
- precisa ordenar registros em memória por um ou vários campos

Evite quando

- implementar Bubble/Selection em produção onde sorted() resolve
- ordenar toda a coleção quando só precisa dos menores k e um heap pode ser melhor
- ordenar em Python grandes conjuntos que o banco poderia ordenar/limitar

Erros e armadilhas comuns

- lista = lista.sort()
- esquecer que key recebe um elemento e devolve a chave
- confundir estabilidade com “não mudar nada”
- ignorar tipos incompatíveis na comparação

No banco lojas_rede

Produtos e estoques podem ser ordenados por preço, nome ou quantidade. O exemplo 08 compara ORDER BY e sorted(); ambos podem gerar o mesmo resultado, mas o local do processamento e o volume transferido são diferentes.

Como ler os exemplos da pasta

Arquivo	O que observar
01_bubble_sort.py	bubble sort
02_bubble_otimizado.py	bubble otimizado
03_selection_sort.py	selection sort
04_insertion_sort.py	insertion sort
05_sort_vs_sorted.py	sort vs sorted
06_key_lambda_multicriterio.py	key lambda multicriterio
07_ordenar_produtos_do_banco.py	ordenar produtos do banco
08_order_by_vs_sorted.py	order by vs sorted

Trecho para leitura — exemplos/15_ordenacao/01_bubble_sort.py

```
valores = [5, 2, 4, 1, 3]
for fim in range(len(valores) - 1, 0, -1):
    for i in range(fim):
        if valores[i] > valores[i + 1]:
            valores[i], valores[i + 1] = valores[i + 1], valores[i]
print(valores)
```

Trecho para leitura — exemplos/15_ordenacao/05_sort_vs_sorted.py

```
precos = [249.90, 39.90, 899.90]
nova = sorted(precos)
print("Original:", precos)
print("sorted:", nova)
precos.sort()
print("sort alterou original:", precos)
```

Trecho para leitura — exemplos/15_ordenacao/07_ordenar_produtos_do_banco.py

```
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("""SELECT p.sku,p.nome,p.preco,e.quantidade FROM produtos p JOIN estoques e ON
e.id_produto=p.id_produto WHERE e.id_unidade=1""")
produtos=cursor.fetchall()
print(sorted(produtos,key=lambda p:(p[3],float(p[2]))[:5]))
cursor.close(); conexao.close()
```

Como pensar nas atividades

- defina a chave de ordenação e o sentido
- decida se deve preservar a coleção original
- escolha banco ou memória conforme onde os dados estão e o volume
- se a atividade pede algoritmo manual, conte comparações/trocas em vez de apenas usar sorted()

Arquivo	Foco	Tipo
01_bubble.py	Bubble Sort	fixacao
02_bubble_flag.py	Bubble com flag	fixacao
03_selection.py	Selection Sort	fixacao
04_insertion.py	Insertion Sort	fixacao
05_algoritmos.py	Comparar resultados	comparacao
06_sort_sorted.py	sort() x sorted()	fixacao
07_key.py	key e reverse	fixacao
08_produtos.py	Produtos por preço	aplicacao com banco
09_clientes.py	Clientes por cidade/nome	aplicacao com banco
10_order_by.py	ORDER BY x sorted	comparacao
11_estabilidade.py	Ordenação estável	conceito
12_contagens.py	Contar comparações/trocas	complexidade

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
no lugar	lista.sort(key=..., reverse=True)
nova lista	sorted(iteravel, key=..., reverse=True)
Bubble/Selection/Insertion	$O(n^2)$ no pior caso
Python sort	$O(n \log n)$ no caso geral
SQL	ORDER BY campo ASC/DESC

16. Recursividade: caso base, pilha de chamadas e hierarquias

Uma função recursiva chama a si mesma sobre um problema menor. Para terminar, precisa de um caso base alcançável. Recursividade combina naturalmente com estruturas hierárquicas como categorias e com cadeias encadeadas, porque cada elemento contém ou aponta para uma parte semelhante do problema.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Caso base e passo recursivo

O caso base resolve diretamente a menor instância e encerra a cadeia de chamadas. O passo recursivo reduz o problema e chama a função novamente. Se o problema não diminui ou o caso base não é alcançado, o Python termina com RecursionError após exceder o limite da pilha.

Pilha de chamadas

Cada chamada mantém parâmetros e variáveis locais até que a chamada interna termine. Isso forma uma pilha implícita. Por isso DFS recursivo e percursos de árvore funcionam sem criar uma list de pilha manualmente; o custo de memória, entretanto, existe.

Categorias pai → filhos

A tabela categorias armazena id_categoria_pai. Após fetchall(), o programa pode construir um dict pai -> lista de filhos. Uma função imprimir(id) mostra a categoria e chama a si mesma para cada filho. Esse padrão é mais natural que escrever um número fixo de laços, porque a profundidade pode variar.

Ciclos e visitados

Uma árvore válida não possui ciclos, mas estruturas gerais podem. Se por erro de dados A aponta para B e B volta a A, recursão simples não termina. Um set de visitados protege percursos em grafos e hierarquias potencialmente cíclicas.

Quando usar

- estrutura é naturalmente definida em termos de subestruturas
- árvores e DFS
- problema recursivo fica mais claro que a versão iterativa

Evite quando

- problemas lineares simples em que um for é mais claro
- profundidades muito grandes que podem exceder a pilha
- estruturas com ciclos sem controle de visitados

Erros e armadilhas comuns

- esquecer caso base
- chamar recursivamente sem reduzir o problema
- recalcular subproblemas desnecessariamente
- não considerar ciclos

No banco lojas_rede

Categorias são a principal fonte real; etapas_reposicao também formam uma cadeia que pode ser percorrida recursivamente. A recursão acontece em memória após a consulta. O fato de uma função visitar uma categoria não altera nenhum registro.

Como ler os exemplos da pasta

Arquivo	O que observar
01_caso_base_contagem.py	caso base contagem
02_somar_lista_recursivamente.py	somar lista recursivamente
03_buscar_em_cadeia_etapas.py	buscar em cadeia etapas
04_categorias_do_banco_mapa_filhos.py	categorias do banco mapa filhos
05_imprimir_arvore_categorias.py	imprimir arvore categorias
06_profundidade_categoria.py	profundidade categoria

Trecho para leitura — exemplos/16_recursividade/01_caso_base_contagem.py

```
def contagem_regressiva(n):
    if n == 0:
        print("Fim")
        return
    print(n)
    contagem_regressiva(n - 1)

contagem_regressiva(5)
```

Trecho para leitura — exemplos/16_rekursividade/05_imprimir_arvore_categorias.py

```

from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()
cursor.execute("SELECT id_categoria, nome, id_categoria_pai FROM categorias")
registros = cursor.fetchall()
cursor.close(); conexao.close()

filhos = {}
for id_categoria, nome, pai in registros:
    filhos.setdefault(pai, []).append((id_categoria, nome))

def imprimir(pai, nivel=0):
    for id_categoria, nome in filhos.get(pai, []):
        print(" " * nivel + nome)
        imprimir(id_categoria, nivel + 1)

imprimir(None)

```

Como pensar nas atividades

- defina primeiro o caso base em português
- mostre como o argumento se aproxima desse caso
- desenhe duas ou três chamadas antes de codificar
- se os dados podem conter ciclos, use visitados

Arquivo	Foco	Tipo
01_caso_base.py	Caso base	fixacao
02_soma.py	Soma recursiva	fixacao
03_fatorial.py	Fatorial	fixacao
04_sem_base.py	Erro sem caso base	conceito
05_mapa_categorias.py	Mapa pai->filhos	aplicacao com banco
06_imprimir_arvore.py	Imprimir categorias	aplicacao com banco
07_profundidade.py	Profundidade	aplicacao com banco
08_descendentes.py	Descendentes	aplicacao com banco
09_recurso_iteracao.py	ETAPAS: ITERAÇÃO X RECURSÃO	escolha de recurso
10_pilha_chamadas.py	Pilha de chamadas	conceito
11_total_arvore.py	Contar nós	aplicacao com banco
12_ciclo.py	Ciclo em estrutura	diagnostico

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
forma	def f(x): if base: return ...; return f(menor)
proteção em grafo	visitados = set()
erro típico	RecursionError
árvore	para cada filho: f(filho)

17. Lista encadeada: nós, referências e uma cadeia persistida

Uma lista simplesmente encadeada é formada por nós. Cada nó guarda um valor e uma referência para o próximo; o último aponta para None. Diferentemente da list nativa, não há acesso direto por índice: chegar ao centésimo nó exige percorrer os anteriores. A implementação é didática e prepara o entendimento de árvores e grafos baseados em objetos/referências.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Classe No

A classe mínima possui valor e proximo. Criar `a.proximo = b` não copia `b`; armazena uma referência ao objeto. Esse é o mesmo conceito de referência estudado com listas, agora exposto explicitamente na estrutura.

Percurso e busca

Começa-se na cabeça e segue-se `atual = atual.proximo` até `None`. Busca é sequencial $O(n)$. Inserir no início é $O(1)$ quando a cabeça é conhecida; inserir no fim exige percorrer $O(n)$ se a estrutura não guarda referência adicional para o último.

Remoção

Para remover um nó intermediário, é necessário fazer o anterior apontar para o próximo do removido. Remover a cabeça é caso especial porque a própria referência inicial muda. Esses casos mostram por que manipular referências exige atenção.

etapas_reposicao como cadeia persistida

A tabela guarda `id_proxima_etapa`, uma chave estrangeira que representa o próximo ID. Depois do `SELECT`, o programa pode percorrer IDs por dict ou criar objetos `No` ligados em memória. Alterar `no.proximo` não altera `id_proxima_etapa` no MySQL; são representações em camadas diferentes.

Quando usar

- estudar estruturas ligadas por referências
- inserções no início frequentes em uma implementação manual
- base conceitual para outras estruturas

Evite quando

- substituir `list` nativa sem motivo prático em Python
- quando precisa acesso aleatório por índice
- quando a clareza da `list` é superior e não há objetivo didático

Erros e armadilhas comuns

- perder a cabeça da lista
- criar ciclo acidental
- não tratar remoção da cabeça
- confundir referência `proximo` em memória com FK persistida

No banco lojas_rede

`etapas_reposicao` contém uma sequência: Receber solicitação → Separar produtos → Expedir no CD → Transportar → Receber na unidade → Concluir reposição. Esse encadeamento dá um exemplo real sem exigir que a lista encadeada inteira seja persistida como objetos Python.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de `SELECT` é local ao processo Python. Operações como `append()`, `pop()`, `add()`, alteração de dict, `popleft()`, `heappop()` ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_no_basico.py	no basico
02_percorrer_nos.py	percorrer nos
03_classe_lista_inserir_fim.py	classe lista inserir fim
04_buscar_no.py	buscar no
05_etapas_reposicao_persistidas.py	etapas reposicao persistidas
06_percorrer_cadeia_do_banco.py	percorrer cadeia do banco
07_objetos_a_partir_da_cadeia_banco.py	objetos a partir da cadeia banco

Trecho para leitura — exemplos/17_lista_encadeada/01_no_basico.py

```
class No:
    def __init__(self, valor):
        self.valor = valor
        self.proximo = None

primeiro = No("Receber pedido")
segundo = No("Validar pagamento")
primeiro.proximo = segundo
print(primeiro.valor)
print(primeiro.proximo.valor)
```

Trecho para leitura — exemplos/17_lista_encadeada/03_classe_lista_inserir_fim.py

```
class No:
    def __init__(self, valor):
        self.valor = valor
        self.proximo = None

class ListaEncadeada:
    def __init__(self):
        self.inicio = None

    def inserir_fim(self, valor):
        novo = No(valor)
        if self.inicio is None:
            self.inicio = novo
            return
        atual = self.inicio
        while atual.proximo is not None:
            atual = atual.proximo
        atual.proximo = novo

lista = ListaEncadeada()
for valor in ["Receber", "Validar", "Separar"]:
    lista.inserir_fim(valor)
print(lista.inicio.proximo.valor)
```

Trecho para leitura — exemplos/17_lista_encadeada/06_percorrer_cadeia_do_banco.py

```
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("SELECT id_etapa,nome,id_proxima_etapa FROM etapas_reposicao")
etapas={id_etapa:(nome,prox) for id_etapa,nome,prox in cursor.fetchall()}
atual=1
while atual is not None:
    nome,atual=etapas[atual]
    print(nome)
cursor.close(); conexao.close()
```

Como pensar nas atividades

- desenhe nós e setas antes de alterar referências
- guarde referências necessárias antes de desconectar um nó
- distinga ID persistido de objeto referenciado
- para acesso por posição, lembre que o percurso começa na cabeça

Arquivo	Foco	Tipo
01_no.py	Classe No	fixacao
02_percorrer.py	Percorrer nós	fixacao
03_inserir_inicio.py	Inserir no início	fixacao
04_inserir_fim.py	Inserir no fim	fixacao
05_buscar.py	Buscar	fixacao
06_remover.py	Remover	fixacao
07_etapas.py	Etapas persistidas	aplicacao com banco

Arquivo	Foco	Tipo
08_objetos.py	Banco para objetos No	transformacao
09_ponteiro.py	REFERÊNCIA EM MEMÓRIA X FK	memória x persistência
10_list_ou_encadeada.py	list x encadeada	escolha de recurso
11_acesso.py	Acesso por posição	complexidade
12_tamanho.py	Tamanho e percurso	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
nó	class No: valor; proximo
percorrer	while atual is not None: atual = atual.proximo
inserir início	novo.proximo = cabeca; cabeca = novo
busca	percurso O(n)

18. Hashing: por que dict e set localizam chaves rapidamente

Hashing transforma uma chave hashable em um valor inteiro usado internamente para escolher posições em uma tabela. dict e set usam esse princípio para oferecer pertencimento e acesso por chave com custo médio próximo de $O(1)$. O capítulo não tenta reimplementar o dicionário completo; o objetivo é entender hash, colisões, chaves válidas e a diferença entre um índice em memória e mecanismos do MySQL.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

hash() e estabilidade

hash(objeto) devolve um inteiro usado pela implementação. Ele não deve ser salvo no banco como identificador persistente. Para alguns tipos, especialmente strings, o Python pode randomizar hashes entre execuções por segurança; o valor concreto não é parte do contrato da aplicação.

Chaves hashable

Uma chave precisa ter hash estável enquanto estiver no dicionário. int, str e tuples de elementos hashable são comuns. list e dict são mutáveis e não podem ser chaves. A imutabilidade da chave evita que ela “mude de posição” depois de inserida.

Colisões

Chaves diferentes podem produzir posições conflitantes. Implementações reais possuem estratégias para resolver colisões. Por isso $O(1)$ é custo médio, não garantia absoluta para toda operação. Um hash didático módulo 5 permite visualizar duas chaves caindo no mesmo bucket.

Índice por SKU e CPF

Carregar produtos e construir sku -> registro ou clientes e construir cpf_cnpj -> dados cria uma tabela hash temporária. É útil para várias buscas. A restrição UNIQUE do MySQL sobre produtos.sku garante unicidade persistida, mas não significa que o dict seja o índice interno do banco.

Quando usar

- buscas repetidas por chave
- pertencimento rápido
- eliminar duplicatas via set
- criar índices temporários depois de fetchall()

Evite quando

- persistir hash() como chave durável
- usar objeto mutável como chave
- construir um índice grande para uma única busca pontual

Erros e armadilhas comuns

- tratar $O(1)$ como garantia matemática de uma única operação
- confundir UNIQUE/índice SQL com dict Python
- esperar o mesmo hash de string entre processos
- não considerar memória adicional

No banco lojas_rede

SKU e CPF/CNPJ são chaves naturais para os exemplos. O banco mantém suas próprias estruturas e restrições; o programa cria outro índice temporário depois de materializar dados. Se o banco muda, o dict não se atualiza até uma nova consulta/reconstrução.

Como ler os exemplos da pasta

Arquivo	O que observar
01_hash_builtin.py	hash builtin
02_dict_indice_por_sku.py	dict indice por sku
03_set_pertencimento.py	set pertencimento
04_colisao_hash_didatica.py	colisao hash didatica
05_indice_produtos_banco.py	indice produtos banco
06_cpf_como_chave_de_dicionario.py	cpf como chave de dicionario

Trecho para leitura — exemplos/18_hashing/01_hash_builtin.py

```
print(hash("CAB-SHA-001"))
print(hash("PERF-FEM-001"))
```

Trecho para leitura — exemplos/18_hashing/05_indice_produtos_banco.py

```
from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

cursor.execute("SELECT sku, id_produto, nome, preco FROM produtos")
por_sku = {sku: (id_produto, nome, preco) for sku, id_produto, nome, preco in cursor.fetchall()}
print(por_sku["PEL-COR-001"])

cursor.close()
conexao.close()
```

Como pensar nas atividades

- escolha uma chave estável e única no conjunto em memória
- avalie quantas buscas justificam construir o índice
- se a origem é o banco, saiba quando o índice fica desatualizado
- se só precisa de uma consulta, considere WHERE

Arquivo	Foco	Tipo
01_hash.py	hash()	fixacao
02_dict.py	Índice por chave	fixacao
03_set.py	Pertencimento	fixacao
04_colisao.py	Colisão didática	conceito
05_sku.py	Índice por SKU	aplicacao com banco
06_cpf.py	Índice por CPF/CNPJ	aplicacao com banco
07_imutabilidade_chave.py	Chaves hashable	conceito
08_unique.py	UNIQUE x hashing	conceito

Arquivo	Foco	Tipo
09_media.py	O(1) médio	complexidade
10_conjuntos_disponibilidade.py	SETS E HASHING	integração
11_duplicata_dict.py	Chave repetida	diagnostico
12_indice_temporario.py	Quando criar índice em memória	escolha de recurso

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
hash	hash(chave)
índice	índice = {registro[chave]: registro for registro in dados}
buscar	índice.get(sku)
pertencimento set	sku in skus
custo	O(1) médio

19. Árvores: hierarquias, árvores binárias e árvores de busca

Árvore organiza elementos em relações pai/filhos. A tabela categorias é uma árvore geral: um pai pode possuir vários filhos. Uma árvore binária limita cada nó a no máximo dois filhos. Uma BST adiciona regra de ordenação: valores menores ficam à esquerda e maiores à direita. Esses conceitos devem ser mantidos separados para não confundir a hierarquia do banco com uma BST.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Vocabulário

Raiz não possui pai; folha não possui filhos; profundidade mede distância da raiz; altura mede o maior caminho até uma folha; subárvore é um nó com todos os seus descendentes. Em categorias, raízes como Cabelos, Perfumaria, Maquiagem, Higiene, Cuidados com a Pele e Acessórios têm filhos armazenados por `id_categoria_pai`.

BST: inserção e busca

Ao inserir, compara-se com o nó atual e segue-se esquerda/direita até espaço vazio. Busca faz o mesmo e pode descartar uma subárvore inteira a cada comparação se a árvore estiver razoavelmente balanceada. Em árvore degenerada, a busca pode cair para $O(n)$.

Percursos

Em ordem visita esquerda, nó, direita e em uma BST válida produz chaves ordenadas. Pré-ordem visita nó antes dos filhos; pós-ordem visita depois. Em árvore geral, a ideia de percurso permanece, mas cada nó percorre uma lista de filhos.

Categorias e BST de preços

A árvore de categorias vem da relação pai/filhos do banco. O exemplo de BST de preços é outra estrutura criada em memória a partir de produtos. Ela não substitui ORDER BY nem cria um índice persistente; serve para estudar propriedade de busca e percurso.

Quando usar

- dados têm hierarquia
- precisa percorrer descendentes/ancestrais
- estudar busca ordenada dinâmica com BST

Evite quando

- usar BST manual em vez de dict/list sem necessidade
- assumir que toda árvore é binária
- esperar eficiência logarítmica de BST desbalanceada

Erros e armadilhas comuns

- confundir raiz com primeiro elemento de uma lista
- tratar categoria geral como BST
- esquecer caso base nos percursos recursivos
- ignorar valores duplicados e regra de inserção

No banco lojas_rede

categorias fornece uma hierarquia verdadeira do domínio. Produtos podem ser inseridos em uma BST temporária por (preço, SKU) para evitar ambiguidade em preços repetidos. Nenhuma dessas estruturas em memória altera categoria, produto ou índice do MySQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_conceitos_arvore_binaria.py	conceitos arvore binaria
02_bst_inserir.py	bst inserir
03_bst_buscar.py	bst buscar
04_percurso_em_ordem.py	percurso em ordem
05_pre_pos_ordem.py	pre pos ordem
06_arvore_categorias_do_banco.py	arvore categorias do banco
07_folhas_categorias.py	folhas categorias
08_bst_precos_produtos_banco.py	bst precos produtos banco

Trecho para leitura — exemplos/19_arvores/02_bst_inserir.py

```
class No:
    def __init__(self, valor):
        self.valor = valor
        self.esquerda = None
        self.direita = None

def inserir(no, valor):
    if no is None:
        return No(valor)
    if valor < no.valor:
        no.esquerda = inserir(no.esquerda, valor)
    elif valor > no.valor:
        no.direita = inserir(no.direita, valor)
    return no

raiz = None
for valor in [50, 30, 80, 20, 40, 70, 90]:
    raiz = inserir(raiz, valor)
print(raiz.esquerda.direita.valor)
```


Trecho para leitura — exemplos/19_arvores/04_percurso_em_ordem.py

```
class No:
    def __init__(self, valor, esquerda=None, direita=None):
        self.valor = valor; self.esquerda = esquerda; self.direita = direita

def em_ordem(no):
    if no is None: return
    em_ordem(no.esquerda)
    print(no.valor)
    em_ordem(no.direita)

raiz = No(50, No(30, No(20), No(40)), No(80, No(70), No(90)))
em_ordem(raiz)
```

Trecho para leitura — exemplos/19_arvores/06_arvore_categorias_do_banco.py

```
from database import criar_conexao

conexao = criar_conexao()
cursor = conexao.cursor()

cursor.execute("SELECT id_categoria, nome, id_categoria_pai FROM categorias")
registros = cursor.fetchall()
filhos = {}
for id_categoria, nome, pai in registros:
    filhos.setdefault(pai, []).append((id_categoria, nome))
for raiz in filhos.get(None, []):
    print("Raiz:", raiz)

cursor.close()
conexao.close()
```

Como pensar nas atividades

- identifique se a árvore é geral ou binária
- defina claramente a regra de filhos
- para BST, defina tratamento de iguais
- escolha percurso conforme o resultado desejado

Arquivo	Foco	Tipo
01_no_binario.py	Nó binário	fixacao
02_bst_insert.py	Inserção em BST	fixacao
03_bst_search.py	Busca em BST	fixacao
04_inorder.py	Em ordem	fixacao
05_pre_pos.py	Pré e pós-ordem	fixacao
06_min_max.py	Mínimo e máximo	fixacao
07_categorias.py	Árvore de categorias	aplicacao com banco
08_folhas.py	Folhas	aplicacao com banco
09_subarvore.py	Subárvore	aplicacao com banco
10_bst_precos.py	BST de preços	aplicacao com banco
11_desbalanceamento.py	Árvore desbalanceada	conceito
12_arvore_hierarquia.py	Quando árvore faz sentido	escolha de recurso

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
em ordem	esquerda → nó → direita
pré-ordem	nó → filhos

Recurso	Forma
pós-ordem	filhos → nó
BST busca média balanceada	$O(\log n)$
BST degenerada	$O(n)$

20. Heap e fila de prioridade: retirar repetidamente o mais urgente

heapq implementa um min-heap sobre list. O menor item fica acessível na raiz e heappop() o remove em $O(\log n)$. Um heap não é uma lista totalmente ordenada; ele garante apenas a propriedade necessária para localizar o mínimo com eficiência. solicitacoes_reposicao já possui prioridade e prazo, tornando o heap uma estrutura natural para processar urgência.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

heapify(), heappush() e heappop()

heapify(lista) reorganiza a list existente em $O(n)$. heappush(heap, x) insere preservando a propriedade em $O(\log n)$. heappop(heap) remove e devolve o menor. heap[0] consulta o menor sem remover.

Tuplas como chave de prioridade

Como tuplas são comparadas lexicograficamente, um item (prioridade, prazo, id, dados) usa prioridade primeiro, prazo como desempate e id como desempate adicional. Incluir um campo comparável antes de objetos complexos evita TypeError quando dois itens empatam nos critérios anteriores.

nsmallest() e nlargest()

Essas funções obtêm alguns extremos e podem ser mais simples que manter manualmente um heap, especialmente para consultas pontuais. Se a intenção é exibir toda a coleção ordenada, sorted() geralmente é mais claro.

FIFO versus heap

Fila responde “quem chegou primeiro?”. Heap responde “quem tem menor chave de prioridade?”. São políticas distintas sobre os mesmos dados. A escolha vem da regra do problema, não da estrutura que o aluno prefere usar.

Quando usar

- precisa retirar repetidamente o menor/mais urgente
- fila de prioridade
- top-k em certos cenários
- algoritmos que usam prioridade

Evite quando

- quando precisa de toda a lista ordenada apenas uma vez
- quando FIFO é a regra correta
- quando uma simples min()/max() única resolve

Erros e armadilhas comuns

- achar que a representação interna do heap está totalmente ordenada
- esquecer desempate de tuplas
- confundir heappop() com UPDATE no banco
- inverter prioridade: no banco 1 é mais urgente que 5

No banco lojas_rede

Os exemplos carregam solicitações PENDENTE e constroem tuplas de prioridade. heappop() apenas retira da estrutura local; o status persistido não muda. Para registrar atendimento, é necessário UPDATE. Isso permite executar fila FIFO e heap sobre a mesma tabela e comparar políticas.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como append(), pop(), add(), alteração de dict, popleft(), heappop() ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_heapify_heappop.py	heapify heappop
02_heappush.py	heappush
03_desempate_tupla.py	desempate tupla
04_reposicoes_banco_para_heap.py	reposicoes banco para heap
05_nsmallest_nlargest.py	nsmallest nlargest
06_fila_prioridade_nao_atualiza_status.py	fila prioridade nao atualiza status

Trecho para leitura — exemplos/20_heap_prioridade/01_heapify_heappop.py

```
import heapq

valores = [5, 1, 4, 2, 3]
heapq.heapify(valores)
print(valores)
print(heapq.heappop(valores))
```

Trecho para leitura — exemplos/20_heap_prioridade/04_reposicoes_banco_para_heap.py

```
import heapq
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("SELECT prioridade,prazo,id_solicitacao,id_unidade,id_produto FROM solicitacoes_reposicao WHERE status='PENDENTE'")
heap=cursor.fetchall(); heapq.heapify(heap)
for _ in range(min(3,len(heap))): print(heapq.heappop(heap))
cursor.close(); conexao.close()
```

Como pensar nas atividades

- defina qual chave significa “mais urgente”
- inclua desempates determinísticos
- decida se precisa dos próximos poucos itens ou de toda a ordenação
- se processar item real, separe seleção em memória de persistência do status

Arquivo	Foco	Tipo
01_heapify.py	heapify	fixacao
02_push_pop.py	heappush/heappop	fixacao
03_tupla_desempate.py	Tuplas como prioridade	fixacao
04_nsmallest.py	nsmallest/nlargest	fixacao
05_tarefas.py	Solicitações de reposição em heap	aplicacao com banco
06_top3.py	Três solicitações mais urgentes	aplicacao com banco
07_pop_nao_update.py	heappop não muda banco	memoria x persistencia
08_fifo_prioridade.py	FILA X HEAP	comparação
09_heap_ou_sorted.py	Heap ou sorted?	escolha de recurso
10_heap_nao_ordenado.py	Heap não é lista ordenada	conceito
11_inserir_nova.py	HEAPPUSH NÃO É INSERT	memória x persistência
12_empate_tipo.py	Desempate incompatível	diagnostico

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
criar heap	heapq.heapify(lista)

Recurso	Forma
inserir	<code>heapq.heappush(heap, item)</code>
retirar menor	<code>heapq.heappop(heap)</code>
consultar menor	<code>heap[0]</code>
k menores	<code>heapq.nsmallest(k, dados, key=...)</code>

21. Grafos: unidades, rotas e representações de conectividade

Grafo modela entidades e relações. No banco, unidades são vértices e rotas ativas são arestas direcionadas. Cada rota possui dois pesos: distância em quilômetros e tempo estimado. Essa modelagem permite estudar lista de adjacência, matriz de adjacência, graus, caminhos e os percursos BFS/DFS dos capítulos seguintes.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Lista de adjacência

Um dict origem -> lista de vizinhos armazena apenas arestas existentes e é eficiente para grafos esparsos. Vizinhos podem ser IDs simples, sets quando duplicação deve ser impedida ou tuples (destino, peso...) em grafo ponderado.

Matriz de adjacência

Uma matriz $n \times n$ marca se existe aresta de i para j . O acesso é direto, mas o consumo de memória cresce $O(n^2)$ mesmo se houver poucas rotas. Como IDs de unidade não são $0..n-1$, o programa precisa mapear ID → índice da matriz.

Direcionamento

Uma linha origem → destino não cria automaticamente destino → origem. O conjunto didático possui várias rotas em ambos os sentidos, mas elas são duas linhas distintas. Essa distinção é necessária para interpretar grau de saída e alcançabilidade.

Pesos

`distancia_km` e `tempo_estimado_min` são dois critérios diferentes. Um caminho com menos arestas pode ter mais quilômetros. A representação do grafo pode conservar ambos; o algoritmo escolhido depois deve saber qual objetivo está resolvendo.

Quando usar

- relações muitos-para-muitos e conectividade
- rotas, redes, dependências e transições
- perguntas de alcançabilidade/caminho

Evite quando

- representar hierarquia simples com grafo genérico se árvore comunica melhor
- usar matriz em grafo grande/esparsos sem necessidade
- perder pesos quando a aplicação precisa deles

Erros e armadilhas comuns

- assumir aresta inversa
- usar ID como índice direto da matriz
- confundir vértice visitado em memória com estado persistido
- chamar qualquer relação de árvore quando há ciclos

No banco lojas_rede

rotas liga lojas e CDs. O exemplo monta adjacência a partir de `SELECT id_origem, id_destino, distancia_km, tempo_estimado_min`. Um dict separado `id_unidade` → nome torna a saída legível. Adicionar um vizinho ao dict não cria uma linha em rotas.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de `SELECT` é local ao processo Python. Operações como `append()`, `pop()`, `add()`, alteração de dict, `popleft()`, `heappop()` ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_lista_adjacencia_literal.py	lista adjacencia literal
02_grafo_com_set.py	grafo com set
03_grafo_ponderado.py	grafo ponderado
04_rotas_banco_para_adjacencia.py	rotas banco para adjacencia
05_nomes_unidades_e_arestas.py	nomes unidades e arestas
06_matriz_adjacencia.py	matriz adjacencia
07_grau_saida.py	grau saida

Trecho para leitura — exemplos/21_grafos/04_rotas_banco_para_adjacencia.py

```
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("SELECT id_origem,id_destino,distancia_km FROM rotas WHERE ativa=1")
grafo={}
for origem,destino,distancia in cursor.fetchall(): grafo.setdefault(origem,[]).append((destino,float(distancia)))
print(grafo[20])
cursor.close(); conexao.close()
```

Trecho para leitura — exemplos/21_grafos/06_matriz_adjacencia.py

```
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("SELECT id_unidade FROM unidades ORDER BY id_unidade")
ids=[x[0] for x in cursor.fetchall()]; pos={v:i for i,v in enumerate(ids)}
matriz=[[0]*len(ids) for _ in ids]
cursor.execute("SELECT id_origem,id_destino FROM rotas WHERE ativa=1")
for o,d in cursor.fetchall(): matriz[pos[o]][pos[d]]=1
print("Unidades:",ids)
print("Primeira linha:",matriz[0])
cursor.close(); conexao.close()
```

Como pensar nas atividades

- identifique vértices e arestas
- decida se o grafo é dirigido e se há pesos
- escolha lista ou matriz conforme densidade e operação
- mantenha mapeamento entre IDs persistidos e índices/objetos em memória

Arquivo	Foco	Tipo
01_adj_list.py	Lista de adjacência	fixacao
02_set_neighbors.py	Vizinhos como set	fixacao
03_ponderado.py	Arestas com peso	fixacao
04_arestas.py	ROTAS PARA LISTA DE ADJACÊNCIA	aplicação com banco
05_nomes.py	IDs E NOMES DAS UNIDADES	aplicação com banco
06_matriz.py	MATRIZ DE ADJACÊNCIA	aplicação com banco
07_grau.py	Grau de saída	fixacao
08_direcionado.py	Direcionado x bidirecional	conceito
09_pesos.py	Dois pesos	conceito
10_lista_matriz.py	Lista x matriz de adjacência	escolha de recurso
11_add_edge.py	ARESTA EM MEMÓRIA NÃO É ROTA NO BANCO	memória x persistência
12_isolados.py	UNIDADES SEM SAÍDA	integração

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
adjacência	<code>grafo.setdefault(origem, []).append(destino)</code>
ponderado	<code>(destino, distancia, tempo)</code>
matriz	<code>matriz[i][j] = 1</code>
grau de saída	<code>len(grafo.get(v, []))</code>

22. BFS: exploração por camadas e caminhos com menos arestas

Breadth-First Search explora primeiro os vizinhos mais próximos em número de arestas. Para isso usa fila: descobre vizinhos, coloca-os no final e processa na ordem em que foram descobertos. Um set de visitados impede repetições em grafos com ciclos. Em grafo não ponderado, BFS encontra caminhos com menor número de arestas.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Fila e visitados

A origem entra na deque e no set visitados. A cada passo, `popleft()` retira o próximo vértice; seus vizinhos ainda não visitados são marcados e enfileirados. Marcar no momento da descoberta, e não apenas ao retirar, evita inserir o mesmo vértice várias vezes.

Distância em arestas

Se `dist[origem] = 0`, cada vizinho recebe `dist[atual] + 1`. Isso mede número de conexões, não quilômetros nem minutos. É adequado para perguntas como “quantas rotas no mínimo são necessárias?” em uma versão não ponderada.

Reconstrução de caminho

Guardar `predecessor[vizinho] = atual` permite reconstruir o caminho ao chegar no destino. Parte-se do destino, segue-se predecessor até a origem e depois inverte-se a lista. Se não houver predecessor/visita, o destino não é alcançável naquela origem.

BFS e pesos

BFS não minimiza `distancia_km` nem `tempo_estimado_min` quando pesos variam. Para caminhos mínimos ponderados são necessários algoritmos apropriados, como Dijkstra em cenários de pesos não negativos, assunto que pode ser estudado posteriormente.

Quando usar

- menor número de arestas em grafo não ponderado
- exploração por níveis
- encontrar todos os alcançáveis
- árvores por nível

Evite quando

- menor distância ponderada com pesos diferentes
- exploração profunda de um ramo como objetivo principal
- executar sem visitados em grafo cíclico

Erros e armadilhas comuns

- usar `list.pop(0)` em vez de deque em implementação eficiente
- marcar visitado tarde demais
- interpretar “menor número de rotas” como “menor km”
- esquecer de reconstruir caminho na ordem correta

No banco lojas_rede

A rede logística possui ciclos e ligações de ida/volta. BFS a partir de Sete Lagoas Centro (1) pode alcançar outras unidades através dos CDs e conexões. O set visitados existe só no algoritmo; nenhuma coluna “visitado” é gravada no banco.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como append(), pop(), add(), alteração de dict, popleft(), heappop() ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_bfs_basico.py	bfs basico
02_bfs_com_distancia_em_arestas.py	bfs com distancia em arestas
03_reconstruir_caminho.py	reconstruir caminho
04_bfs_rotas_banco.py	bfs rotas banco
05_bfs_nao_considera_peso.py	bfs nao considera peso

Trecho para leitura — exemplos/22_bfs/01_bfs_basico.py

```
from collections import deque

grafo = {1:[2,3],2:[1,5],3:[1,4],4:[3],5:[2]}
fila = deque([1])
visitados = {1}
while fila:
    atual = fila.popleft()
    print(atual)
    for vizinho in grafo.get(atual, []):
        if vizinho not in visitados:
            visitados.add(vizinho)
            fila.append(vizinho)
```

Trecho para leitura — exemplos/22_bfs/04_bfs_rotas_banco.py

```
from collections import deque
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("SELECT id_origem,id_destino FROM rotas WHERE ativa=1")
grafo={}
for o,d in cursor.fetchall(): grafo.setdefault(o,[]).append(d)
cursor.close(); conexao.close()
fila=deque([1]); visitados={1}; ordem=[]
while fila:
    atual=fila.popleft(); ordem.append(atual)
    for v in grafo.get(atual,[]):
        if v not in visitados: visitados.add(v); fila.append(v)
print(ordem)
```

Como pensar nas atividades

- remova os pesos se a pergunta é apenas número de conexões
- prepare adjacência e nomes antes do percurso
- use deque e set
- se precisa caminho, guarde predecessor desde o início

Arquivo	Foco	Tipo
01_ordem.py	BFS básico	fixacao
02_distancia.py	Distância em arestas	fixacao
03_caminho.py	Reconstruir caminho	fixacao
04_rotas.py	BFS NA REDE LOGÍSTICA	aplicação com banco
05_caminho.py	CAMINHO EM NÚMERO DE ROTAS	aplicação com banco
06_peso.py	BFS e pesos	conceito
07_fila.py	Por que fila?	conceito

Arquivo	Foco	Tipo
08_visitados.py	Sem visitados	diagnostico
09_bfs_dfs.py	BFS ou DFS?	escolha de recurso
10_alcancaveis.py	UNIDADES ALCANÇÁVEIS	integração
11_visitar_nao_update.py	Visitar não persiste	memoria x persistencia
12_bfs_funcao.py	Função bfs	funcoes

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
fila	deque([origem])
visitados	{origem}
retirar	atual = fila.popleft()
descobrir	visitados.add(v); fila.append(v)
caminho	predecessor[v] = atual

23. DFS: exploração em profundidade, pilha e recursão

Depth-First Search segue um ramo o máximo possível antes de voltar. Pode ser implementada iterativamente com uma pilha ou recursivamente usando a pilha de chamadas. Assim como BFS, precisa de visitados em grafos com ciclos. DFS é excelente para exploração completa, componentes, detecção de alcance e percursos de hierarquias.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Versão iterativa

A pilha começa com a origem. `pop()` retira o topo; se ainda não foi visitado, marca e empilha seus vizinhos. A ordem final depende da ordem em que vizinhos são empilhados; `reversed()` é frequentemente usado quando se deseja imitar a ordem natural de uma versão recursiva.

Versão recursiva

A função marca o atual e chama `dfs(vizinho)` para cada vizinho não visitado. A simplicidade combina com árvores/categorias, mas grafos muito profundos podem atingir o limite de recursão.

Alcance e componentes

Para saber se destino é alcançável, DFS pode encerrar quando encontra o alvo. Para componentes em grafo não direcionado, percorre-se todos os vértices e inicia-se um novo DFS sempre que aparece um ainda não visitado.

DFS versus BFS

Ambos visitam os vértices alcançáveis, mas a ordem e propriedades diferem. BFS é a escolha natural para menor número de arestas; DFS para explorar ramos, topologias e muitas análises estruturais. A escolha deve seguir a pergunta.

Quando usar

- explorar completamente um ramo
- verificar alcançabilidade
- percorrer árvores/hierarquias
- encontrar componentes ou analisar estrutura

Evite quando

- menor número de arestas como requisito

- recursão em profundidade potencialmente enorme sem controle
- grafo cíclico sem visitados

Erros e armadilhas comuns

- esquecer set de visitados
- confundir ordem de visita com um único resultado obrigatório
- recursão infinita em ciclo
- assumir que DFS acha caminho mais curto

No banco_lojas_rede

DFS sobre rotas percorre a mesma rede usada no BFS, permitindo comparar ordens. Categorias também podem ser vistas como grafo dirigido pai → filhos e percorridas por DFS. Em ambos os casos, visitados e pilha são temporários.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como append(), pop(), add(), alteração de dict, popleft(), heappop() ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_dfs_iterativo.py	dfs iterativo
02_dfs_recurso.py	dfs recursivo
03_detectar_alcance.py	detectar alcance
04_dfs_rotas_banco.py	dfs rotas banco
05_dfs_em_arvore_categorias.py	dfs em arvore categorias

Trecho para leitura — exemplos/23_dfs/01_dfs_iterativo.py

```
grafo = {1:[2,3],2:[1,5],3:[1,4],4:[3],5:[2]}
pilha = [1]; visitados = set()
while pilha:
    atual = pilha.pop()
    if atual in visitados: continue
    visitados.add(atual); print(atual)
    pilha.extend(reversed(grafo.get(atual, [])))
```

Trecho para leitura — exemplos/23_dfs/02_dfs_recurso.py

```
grafo = {1:[2,3],2:[1,5],3:[1,4],4:[3],5:[2]}
visitados = set()

def dfs(atual):
    if atual in visitados: return
    visitados.add(atual); print(atual)
    for v in grafo.get(atual, []): dfs(v)

dfs(1)
```

Trecho para leitura — exemplos/23_dfs/04_dfs_rotas_banco.py

```
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor(); cursor.execute("SELECT id_origem,id_destino FROM rotas
WHERE ativa=1")
grafo={}
for o,d in cursor.fetchall(): grafo.setdefault(o,[]).append(d)
cursor.close(); conexao.close()
visitados=set(); pilha=[1]; ordem=[]
while pilha:
    atual=pilha.pop()
    if atual in visitados: continue
    visitados.add(atual); ordem.append(atual)
    pilha.extend(reversed(grafo.get(atual,[])))
print(ordem)
```

Como pensar nas atividades

- decida iterativa ou recursiva
- defina quando marcar visitado
- se ordem importa, controle a ordem dos vizinhos
- não use DFS para uma propriedade que depende de caminho mínimo em arestas

Arquivo	Foco	Tipo
01_iterativo.py	DFS iterativo	fixacao
02_recursivo.py	DFS recursivo	fixacao
03_alcanca.py	Teste de alcance	fixacao
04_rotas.py	DFS NA REDE LOGÍSTICA	aplicação com banco
05_categorias.py	DFS em categorias	aplicacao com banco
06_pilha.py	Por que pilha?	conceito
07_ciclos.py	CICLOS E VISITADOS	diagnóstico
08_dfs_bfs.py	DFS ou BFS?	escolha de recurso
09_componentes.py	Componentes	integração
10_dfs_ordem.py	Retornar ordem	funcoes
11_set_visitados.py	Visitados em memória	memoria x persistencia
12_ordens.py	Ordens diferentes	comparacao

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
iterativo	pilha = [origem]; atual = pilha.pop()
recursivo	dfs(v): visitados.add(v); ... dfs(vizinho)
visitados	set()
alcance	encerrar ao encontrar destino

24. Integração: escolher estruturas e combinar banco, memória e algoritmos

O objetivo final não é usar toda estrutura em todo problema. É reconhecer propriedades do problema e escolher a representação adequada. Os exemplos de integração combinam consulta ao MySQL, materialização em tuples/lists, índices com dict, conjuntos, filas, ordenação e grafos. A solução boa mantém claro onde cada estado existe e por que cada estrutura foi escolhida.

ANTES DE EXECUTAR — Tente prever a saída dos exemplos básicos. A previsão força você a acompanhar tipos, referências e alterações de estado em vez de apenas observar o resultado.

Pipeline banco → memória → decisão

Uma aplicação típica executa SELECT para obter apenas os dados necessários, recebe tuples/lists, transforma em estruturas mais adequadas ao processamento e produz uma decisão ou saída. Se a decisão precisa alterar o sistema, executa SQL explícito e confirma a transação. Nem toda transformação precisa voltar ao banco.

Catálogo e índices

Um catálogo pode chegar como list de tuples, ganhar um dict por SKU para buscas repetidas e ser ordenado por preço para exibição. Não existe contradição em usar várias estruturas: cada uma atende uma etapa diferente do pipeline.

Reposição por FIFO ou prioridade

As mesmas solicitacoes_reposicao podem ser interpretadas como deque quando a política é chegada ou heap quando a política é urgência. O dado é o mesmo; a regra de processamento muda a estrutura.

Disponibilidade e set

Estoques de duas unidades podem virar sets de SKUs para interseção/diferença. Depois, um dict por SKU pode recuperar os dados completos de cada produto. Combinar estruturas evita forçar uma única representação a fazer tudo.

Rotas e BFS

rotas viram lista de adjacência; unidades viram dict id → nome; BFS encontra um caminho em número de arestas. Cada transformação adiciona uma capacidade específica e permanece local.

Persistência explícita

Se o sistema precisa marcar uma solicitação como EM_SEPARACAO, reduzir estoque ou criar pedido, é necessário voltar à camada SQL. O ponto de controle é consciente: UPDATE/INSERT/DELETE + commit, ou rollback em caso de falha.

Quando usar

- projetar soluções completas
- escolher entre coleções/estruturas com base no problema
- separar consulta, processamento e persistência

Evite quando

- usar uma estrutura porque foi a última estudada
- trazer todo o banco para memória sem necessidade
- misturar regras de persistência com operações locais sem distinguir efeitos

Erros e armadilhas comuns

- não documentar por que escolheu a estrutura
- manter índices em memória depois que o banco mudou sem reconstruir
- confundir ordem do banco com ordem da estrutura
- persistir parcialmente uma operação composta

No banco lojas_rede

O banco lojas_rede foi propositalmente mantido pequeno: clientes, produtos, categorias, unidades, estoques, canais, pedidos, histórico, reposição, etapas e rotas. Ele oferece variedade suficiente para as estruturas sem virar um ERP completo. A integração final deve usar esse domínio como contexto e preservar o princípio “o objeto Python não é a linha do MySQL”.

MEMÓRIA X PERSISTÊNCIA — Toda estrutura criada depois de SELECT é local ao processo Python. Operações como append(), pop(), add(), alteração de dict, popleft(), heappop() ou marcação de visitados não executam SQL.

Como ler os exemplos da pasta

Arquivo	O que observar
01_catalogo_indice_hash_e_ordenacao.py	catalogo indice hash e ordenacao
02_fila_reposicao_com_persistencia_explicita.py	fila reposicao com persistencia explicita
03_disponibilidade_por_intersecao.py	disponibilidade por intersecao
04_estoque_baixo_por_unidade.py	estoque baixo por unidade
05_rota_bfs_e_nomes.py	rota bfs e nomes
06_pipeline_banco_memoria_decisao.py	pipeline banco memoria decisao

Trecho para leitura — exemplos/24_integracao/01_catalogo_indice_hash_e_ordenacao.py

```
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("""SELECT p.sku,p.nome,p.preco,e.quantidade FROM produtos p JOIN estoques e ON
e.id_produto=p.id_produto WHERE e.id_unidade=1""")
produtos=cursor.fetchall(); por_sku={p[0]:p for p in produtos}
print("Busca média O(1) por chave:",por_sku["PEL-COR-001"])
print("Menores estoques:",sorted(produtos,key=lambda p:p[3])[:3])
cursor.close(); conexao.close()
```

Trecho para leitura — exemplos/24_integracao/02_fila_reposicao_com_persistencia_explicita.py

```
from collections import deque
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("SELECT id_solicitacao,id_unidade,id_produto FROM solicitacoes_reposicao WHERE status='PENDENTE'
ORDER BY data_solicitacao")
fila=deque(cursor.fetchall()); proxima=fila.popleft()
print("Selecionada em memória:",proxima)
print("Persistir exigiria: UPDATE solicitacoes_reposicao SET status='EM_SEPARACAO' WHERE id_solicitacao
=",proxima[0])
cursor.close(); conexao.close()
```

Trecho para leitura — exemplos/24_integracao/05_rota_bfs_e_nomes.py

```
from collections import deque
from database import criar_conexao
conexao=criar_conexao(); cursor=conexao.cursor()
cursor.execute("SELECT id_unidade,nome FROM unidades"); nomes=dict(cursor.fetchall())
cursor.execute("SELECT id_origem,id_destino FROM rotas WHERE ativa=1"); grafo={}
for o,d in cursor.fetchall(): grafo.setdefault(o,[]).append(d)
cursor.close(); conexao.close()
origem,destino=1,9
fila=deque([origem]); anterior={origem:None}
while fila:
    atual=fila.popleft()
    if atual==destino: break
    for v in grafo.get(atual,[]):
        if v not in anterior: anterior[v]=atual; fila.append(v)
if destino in anterior:
    caminho=[]; atual=destino
    while atual is not None: caminho.append(nomes[atual]); atual=anterior[atual]
    print(" -> ".join(reversed(caminho)))
else: print("Sem caminho")
```

Como pensar nas atividades

- comece pela pergunta do problema, não pelo nome da estrutura
- liste propriedades necessárias: ordem, unicidade, chave, FIFO, LIFO, prioridade, hierarquia, conectividade
- busque no banco apenas o recorte necessário
- transforme em memória com intenção explícita
- persista somente quando a regra exigir e delimite a transação

Arquivo	Foco	Tipo
01_catalogo.py	Catálogo em memória	integração
02_disponibilidade.py	DISPONIBILIDADE ENTRE LOJAS	integração
03_fila.py	FILA DE REPOSIÇÃO	integração
04_prioridade.py	Reposição por prioridade	integração
05_arvore.py	Categorias	integração
06_grafo.py	ROTAS LOGÍSTICAS	integração
07_transacao.py	Venda simulada	integração
08_where_python.py	Onde processar?	decisão
09_estrutura.py	ESCOLHER A ESTRUTURA	decisão
10_memoria_persistencia.py	Memória x persistência	síntese
11_painel.py	PAINEL OPERACIONAL TEXTUAL	projeto curto

Arquivo	Foco	Tipo
12_auditoria.py	Auditoria de pedido	projeto curto

NÃO TRANSFORME A ATIVIDADE EM CÓPIA — Use os exemplos para reconhecer o padrão, mas escreva a solução a partir do enunciado. Se você só consegue resolver copiando um arquivo inteiro, volte à explicação e tente descrever em palavras a estrutura necessária.

Consulta rápida

Recurso	Forma
sequência mutável	list
registro posicional	tuple
chave → valor	dict
únicos	set
LIFO	list como pilha
FIFO	deque
prioridade	heapq
hierarquia	árvore
conectividade	grafo
menor nº arestas	BFS
profundidade	DFS

Apêndice A — Dados de referência do banco

As atividades podem ser resolvidas por consulta, mas conhecer os principais IDs torna a leitura dos exemplos mais rápida. Estes dados correspondem ao arquivo database/dados_iniciais.sql do pacote revisado.

Clientes

ID	Nome	Cidade/UF
1	Ana Martins	Sete Lagoas/MG
2	Bruno Alves	Sete Lagoas/MG
3	Carla Souza	Belo Horizonte/MG
4	Diego Lima	Contagem/MG
5	Elisa Rocha	Divinópolis/MG
6	Felipe Nunes	Montes Claros/MG
7	Gabriela Moraes	Uberlândia/MG
8	Henrique Costa	Juiz de Fora/MG
9	Isabela Freitas	Sao Paulo/SP
10	Joao Ribeiro	Brasília/DF

Produtos

ID	SKU	Produto	Preço
1	CAB-SHA-001	Shampoo Nutritivo 350 ml	R\$ 24.90
2	CAB-CON-001	Condicionador Nutritivo 350 ml	R\$ 27.90
3	CAB-MAS-001	Mascara Capilar 300 g	R\$ 34.90
4	PERF-FEM-001	Perfume Floral 100 ml	R\$ 129.90
5	PERF-MAS-001	Perfume Amadeirado 100 ml	R\$ 139.90
6	MAQ-LAB-001	Batom Matte Vermelho	R\$ 29.90
7	PEL-COR-001	Hidratante Corporal 400 ml	R\$ 32.90
8	PEL-FAC-001	Protetor Solar Facial FPS 60	R\$ 59.90
9	HIG-BUC-001	Creme Dental 90 g	R\$ 8.90

ID	SKU	Produto	Preço
10	HIG-COR-001	Desodorante Aerosol 150 ml	R\$ 14.90
11	HIG-BUC-002	Escova Dental Macia	R\$ 12.90
12	PEL-COR-002	Sabonete Liquido 250 ml	R\$ 18.90
13	PEL-FAC-002	Sabonete Facial 200 ml	R\$ 26.90
14	MAQ-ROS-001	Base Liquida Bege 30 ml	R\$ 44.90
15	CAB-SHA-002	Shampoo Anticaspa 350 ml	R\$ 28.90
16	PERF-INF-001	Colonia Infantil 100 ml	R\$ 69.90
17	MAQ-ROS-002	Po Compacto Bege	R\$ 36.90
18	MAQ-OLH-001	Mascara de Cílios	R\$ 39.90
19	ACE-ALG-001	Algodao em Discos 50 un	R\$ 11.90
20	ACE-ESC-001	Escova de Cabelo	R\$ 22.90

Solicitações de reposição

Prioridade 1 é a mais urgente; 5 é a menos urgente. A mesma tabela é usada para comparar FIFO (data_solicitacao) e prioridade (prioridade/prazo).

ID	Unidade	Produto	Qtd.	Pri.	Status	Solicitada em
1	1	4	12	1	PENDENTE	08-28 08:05
2	1	8	18	2	PENDENTE	08-28 08:12
3	2	3	20	3	PENDENTE	08-28 08:18
4	3	14	15	1	EM_SEPARACAO	08-28 08:25
5	4	6	24	2	PENDENTE	08-28 08:31
6	5	1	30	4	PENDENTE	08-28 08:40
7	6	10	25	2	PENDENTE	08-28 08:45
8	7	12	20	1	PENDENTE	08-28 08:55
9	8	5	10	3	PENDENTE	08-28 09:02
10	9	7	18	4	PENDENTE	08-28 09:08
11	10	9	40	2	PENDENTE	08-28 09:14
12	1	15	16	3	PENDENTE	08-28 09:20
13	2	18	12	1	PENDENTE	08-28 09:26
14	5	20	15	5	PENDENTE	08-28 09:35

Rotas logísticas

Origem	Destino	Km	Min
CD Contagem	Sete Lagoas Centro	74.0	70
Sete Lagoas Centro	CD Contagem	74.0	70
CD Contagem	Belo Horizonte Centro	18.0	30
Belo Horizonte Centro	CD Contagem	18.0	30
CD Contagem	Contagem Eldorado	8.0	18
Contagem Eldorado	CD Contagem	8.0	18
CD Contagem	Betim Centro	24.0	35
Betim Centro	CD Contagem	24.0	35
CD Contagem	Divinópolis Centro	120.0	105
Divinópolis Centro	CD Contagem	120.0	105
CD Contagem	Juiz de Fora Centro	285.0	250
Juiz de Fora Centro	CD Contagem	285.0	250
CD Uberlândia	Uberlândia Centro	12.0	20
Uberlândia Centro	CD Uberlândia	12.0	20
CD Uberlândia	Montes Claros Centro	425.0	360
Montes Claros Centro	CD Uberlândia	425.0	360
CD Contagem	CD Uberlândia	535.0	450

Origem	Destino	Km	Min
CD Uberlândia	CD Contagem	535.0	450
CD Contagem	Sao Paulo Centro	590.0	500
Sao Paulo Centro	CD Contagem	590.0	500
CD Uberlândia	Brasília Centro	420.0	360
Brasília Centro	CD Uberlândia	420.0	360
Belo Horizonte Centro	Sete Lagoas Centro	72.0	65
Sete Lagoas Centro	Belo Horizonte Centro	72.0	65
Contagem Eldorado	Betim Centro	20.0	30
Betim Centro	Contagem Eldorado	20.0	30
Uberlândia Centro	Brasília Centro	350.0	300
Brasília Centro	Uberlândia Centro	350.0	300

Apêndice B — Complexidade das operações mais usadas

Estrutura/operação	Custo típico	Observação
list[i]	$O(1)$	acesso por índice
list.append(x)	$O(1)$ amortizado	pode realocar internamente em alguns crescimentos
list.pop()	$O(1)$	remove no final
list.pop(0)	$O(n)$	desloca elementos; para fila use deque
x in list	$O(n)$	busca linear
dict[chave] / chave in dict	$O(1)$ médio	hashing; colisões existem
x in set	$O(1)$ médio	hashing
deque.append / popleft	$O(1)$	operações nas extremidades
heapq.heappush / heappop	$O(\log n)$	fila de prioridade
busca linear	$O(n)$	não exige ordenação
busca binária	$O(\log n)$	exige dados ordenados
Bubble / Selection / Insertion	$O(n^2)$ pior caso	algoritmos didáticos
sorted / list.sort	$O(n \log n)$ típico	Timsort do Python
percorrer árvore/gráfo	$O(V+E)$ em representação por adjacência	quando visita cada vértice/aresta uma vez

Apêndice C — Guia de escolha de estrutura

Se o problema exige...	Considere	Pergunta de verificação
registro posicional estável	tuple	os campos serão apenas lidos e a ordem é conhecida?
sequência mutável e ordenada	list	preciso inserir/remover e preservar ordem?
chave → valor	dict	há uma chave natural para acesso?
elementos únicos	set	repetições são irrelevantes e pertencimento importa?
último a entrar, primeiro a sair	pilha	o evento mais recente deve ser processado primeiro?
primeiro a entrar, primeiro a sair	deque	a ordem de chegada determina o próximo?
menor prioridade primeiro	heapq	preciso retirar repetidamente o mais urgente?
hierarquia	árvore	há relação pai/filhos?
conectividade	gráfo	um elemento pode se ligar a vários outros?
menor número de arestas	BFS	o gráfo é tratado como não ponderado para essa pergunta?
exploração profunda/alcançabilidade	DFS	quero seguir ramos e visitar todos os alcançáveis?

Apêndice D — Regra de ouro para o banco

SELECT COPIA VALORES PARA O PROGRAMA — `fetchone()` e `fetchall()` materializam dados em objetos Python. Depois disso, a estrutura pode ser alterada livremente sem que o MySQL saiba.

PERSISTIR EXIGE SQL — `INSERT` cria linha, `UPDATE` altera, `DELETE` remove. `commit()` confirma a transação. `rollback()` desfaz alterações ainda não confirmadas no banco, mas não restaura objetos Python já modificados.

Quando houver dúvida durante uma atividade, faça um novo `SELECT` e compare o estado persistido com o objeto em memória. Essa verificação é mais confiável do que inferir o estado do banco a partir da variável Python.